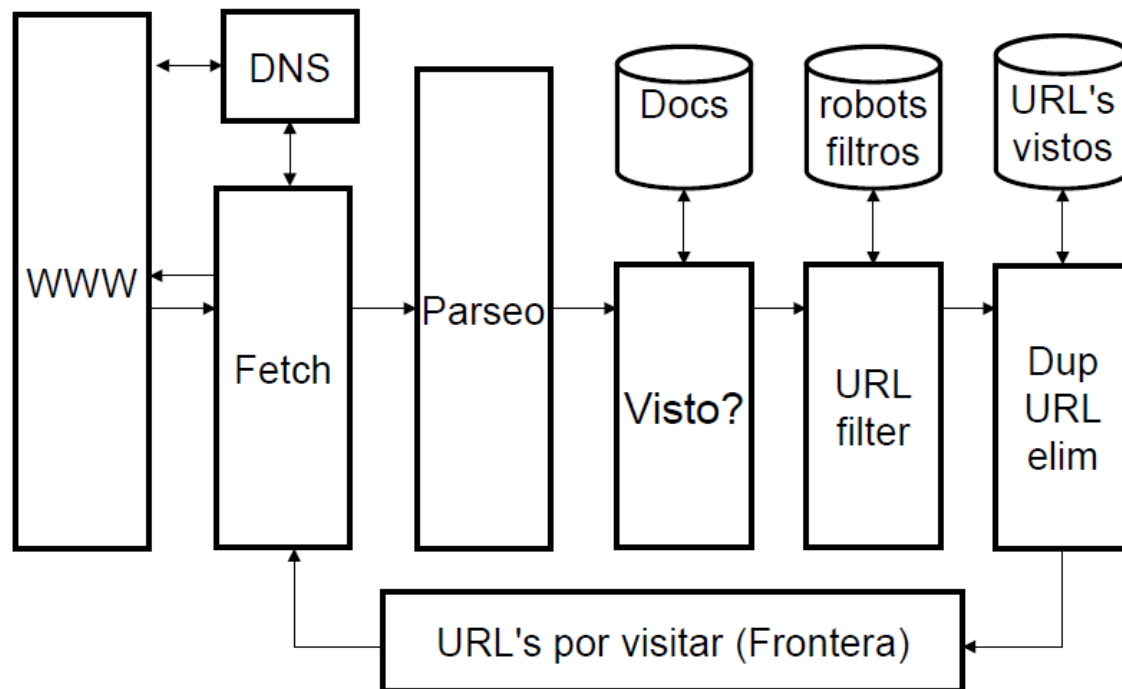


Camino a Hadoop – CAP & BASE

Adaptado de versión original de Aidan
Hogan

¿Cómo se recolectan los documentos en Internet?

- Utilizando Crawlers (arañas, bot o indexadores).
 - Bajan documentos (texto, imágenes, sonidos o videos).
 - Obtienen los URL's para explorar o indexar su contenido.

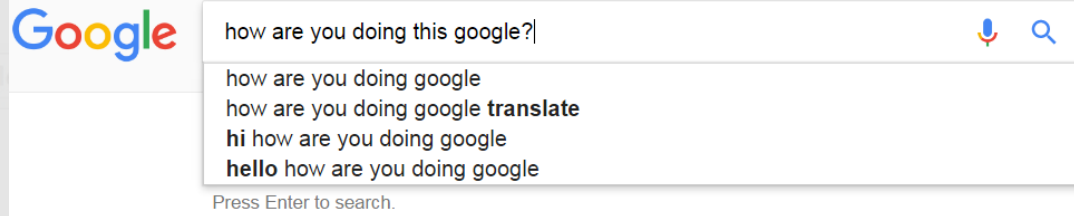


Rastreo de sitios

- Existen mecanismos para sitios que no desean ser rastreados. Por ejemplo, incluir un archivo robots.txt puede solicitar indexar solo partes de un sitio web, o nada en absoluto.

```
User-agent: *  
Disallow: /private/  
Disallow: /confidential/  
Disallow: /other/  
Allow: /other/public/  
  
User-agent: FavoredCrawler  
Disallow:
```

Implementando la búsqueda de Google



¿Qué procesos y algoritmos necesita Google para implementar su búsqueda de la Web?

Crawling



1. Parsear enlaces de las páginas
2. Ordenar los enlaces para bajar
3. Bajar páginas.

Indexación



1. Parsear keywords de las páginas
2. Indexear sus keywords
3. Administrar actualizaciones

Ranking



1. ¿Cuán relevante es una página?
2. ¿Cuán importante es?
3. ¿Cuántos clics tiene?

...



Implementando la búsqueda de Google

Google

how are you doing this google?

how are you doing google

how are you doing google translate

hi how are you doing google

hello how are you doing google

Google

≈ 100 PB / día

≈ (2018) 1,000,000,000,000 URLs

únicas indexadas

1. Parsear enlaces de las páginas

2. Ordenar los enlaces para bajar

3. Bajar páginas, HTML

3. Administrar actualizaciones

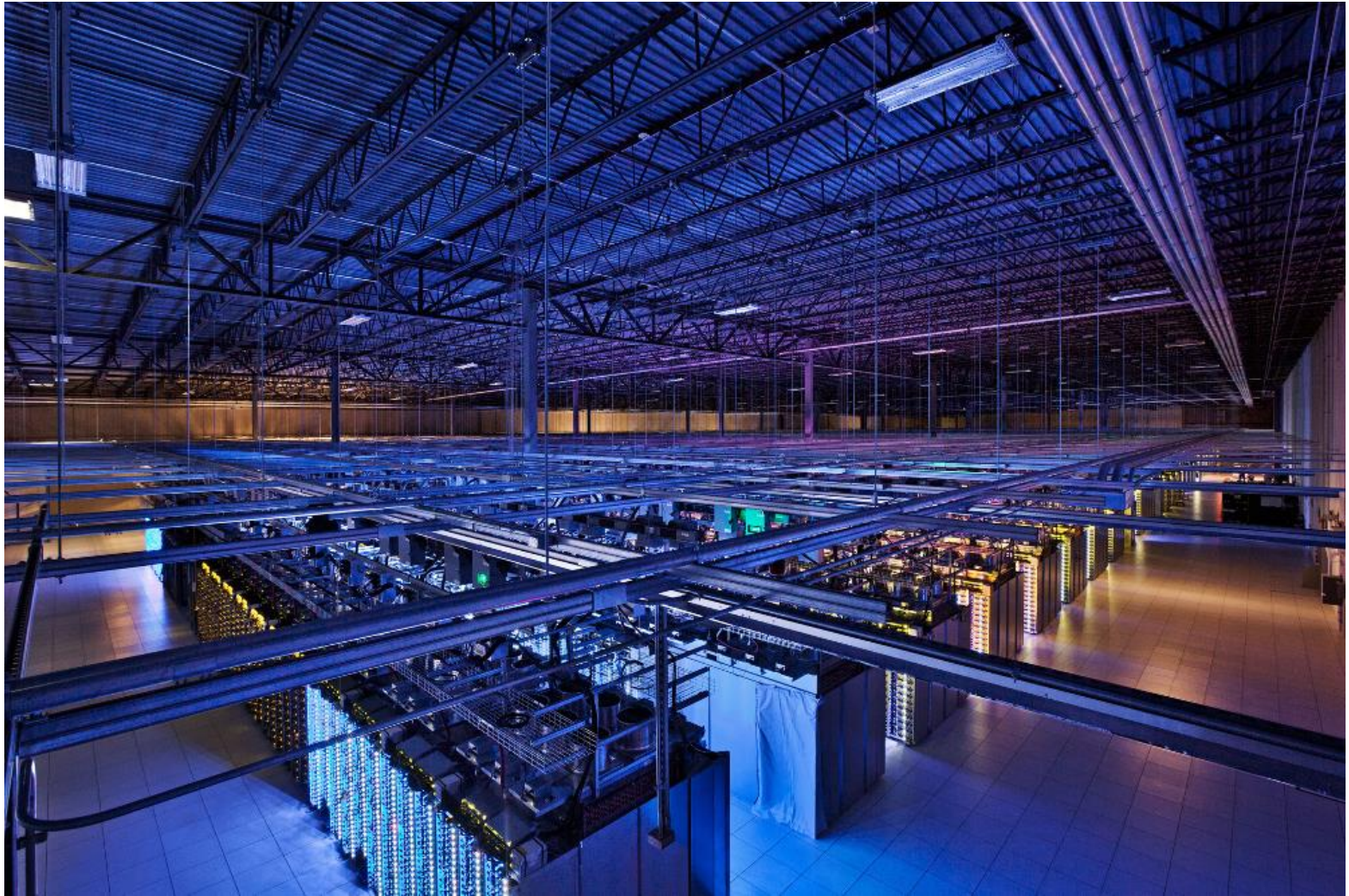
Ranking

1. ¿Cuán relevante es una página?

2. ¿Cuán importante es?

3. ¿Cuántos clics tiene?

Implementando la búsqueda de Google



Implementando la búsqueda de Google

¿Qué abstracciones distribuidas podemos considerar para facilitar aplicaciones como Google?



Crawling

1. Parsear enlaces de las páginas
2. Ordenar los enlaces para bajar
3. Bajar páginas.

Indexación

1. Parsear keywords de las páginas
2. Indexear sus keywords
3. Administrar actualizaciones

Ranking

1. ¿Cuán relevante es una página?
2. ¿Cuán importante es?
3. ¿Cuántos clics tiene?

...

- `write(file f)`
- `read(file f)`
- `delete(file f)`
- `append(file f, data d)`



... al menos para empezar

GOOGLE FILE SYSTEM (GFS)

(SISTEMA DE ARCHIVOS DE GOOGLE)

Google File System (GFS): [Paper](#)

The Google File System

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung

Google*

ABSTRACT

We have designed and implemented the Google File System, a scalable distributed file system for large distributed data-intensive applications. It provides fault tolerance while running on inexpensive commodity hardware, and it delivers high aggregate performance to a large number of clients.

While sharing many of the same goals as previous distributed file systems, our design has been driven by observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure from some earlier file system assumptions. This has led us to reexamine traditional choices and explore radically different design points.

The file system has successfully met our storage needs. It is widely deployed within Google as the storage platform for the generation and processing of data used by our service as well as research and development efforts that require large data sets. The largest cluster to date provides hundreds of terabytes of storage across thousands of disks on over a thousand machines, and it is concurrently accessed by hundreds of clients.

In this paper, we present file system interface extensions designed to support distributed applications, discuss many aspects of our design, and report measurements from both micro-benchmarks and real world use.

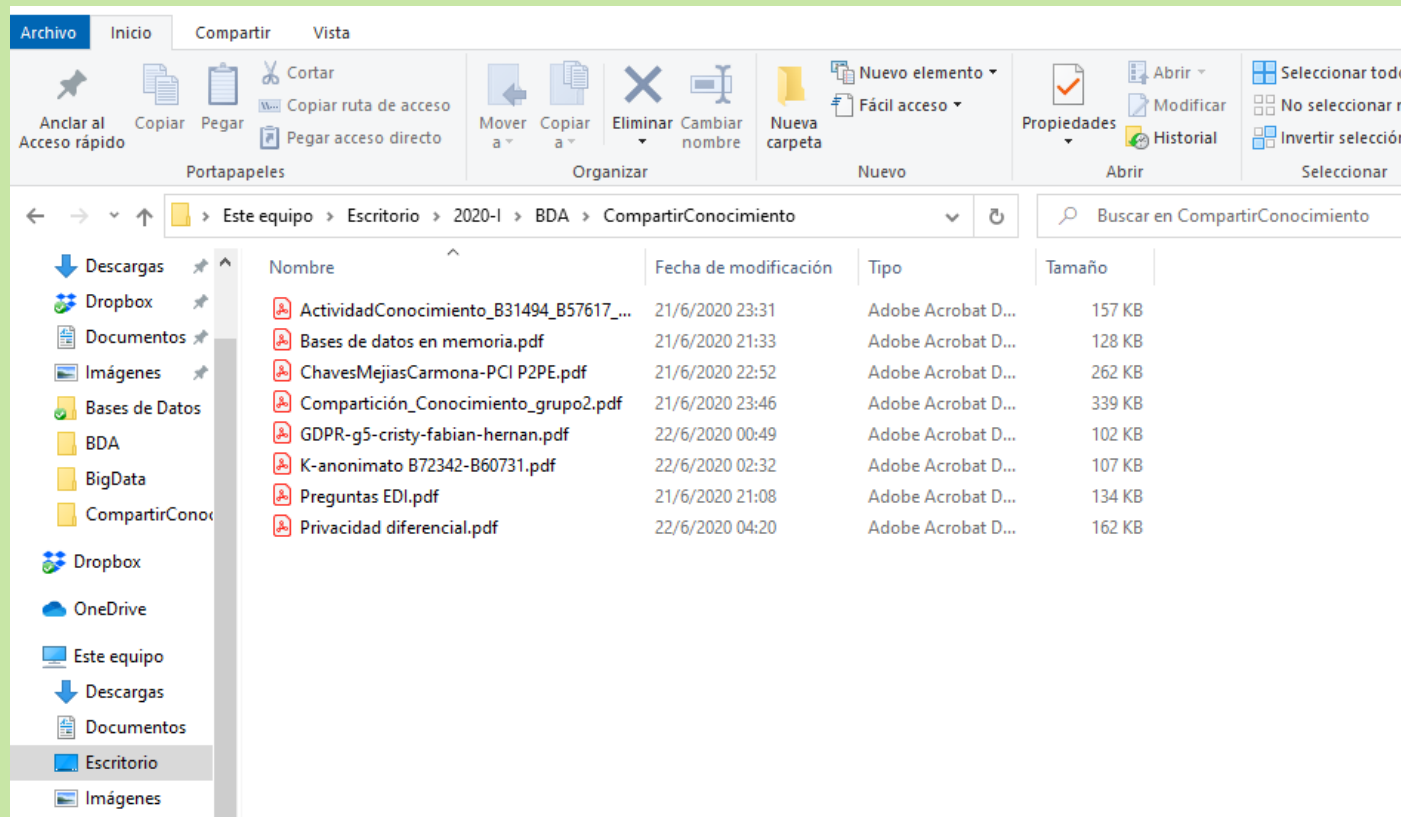
1. INTRODUCTION

We have designed and implemented the Google File System (GFS) to meet the rapidly growing demands of Google's data processing needs. GFS shares many of the same goals as previous distributed file systems such as performance, scalability, reliability, and availability. However, its design has been driven by key observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure from some earlier file system design assumptions. We have reexamined traditional choices and explored radically different points in the design space.

First, component failures are the norm rather than the exception. The file system consists of hundreds or even thousands of storage machines built from inexpensive commodity parts and is accessed by a comparable number of client machines. The quantity and quality of the components virtually guarantee that some are not functional at any given time and some will not recover from their current failures. We have seen problems caused by application bugs, operating system bugs, human errors, and the failures of disks, memory, connectors, networking, and power supplies. Therefore, constant monitoring, error detection, fault tolerance, and automatic recovery must be integral to the system.

Google File System

¿Qué es un sistema de archivos (un “file-system”)?



Google File System

¿Qué es un sistema de archivos (un "file-system")?



 Símbolo del sistema

(c) 2019 Microsoft Corporation. Todos los derechos reservados.

C:\Users\lgesquivel>cd Desktop

C:\Users\lgesquivel\Desktop>cd 2020-I

C:\Users\lgesquivel\Desktop\2020-I>cd BDA

C:\Users\lgesquivel\Desktop\2020-I\BDA>cd CompartirConocimiento

C:\Users\lgesquivel\Desktop\2020-I\BDA\CompartirConocimiento>dir

El volumen de la unidad C no tiene etiqueta.

El número de serie del volumen es: 3891-5A98

Directorio de C:\Users\lgesquivel\Desktop\2020-I\BDA\CompartirConocimiento

22/06/2020 04:20 <DIR> .

22/06/2020 04:20 <DIR> ..

21/06/2020 23:31 159 747 ActividadConocimiento_B31494_B57617_B60259.pdf

21/06/2020 21:33 130 577 Bases de datos en memoria.pdf

21/06/2020 22:52 268 053 ChavesMejiasCarmona-PCI P2PE.pdf

21/06/2020 23:46 347 114 Compartición_Conocimiento_grupo2.pdf

22/06/2020 00:49 103 965 GDPR-g5-cristy-fabian-hernan.pdf

22/06/2020 02:32 109 331 K-anonimato B72342-B60731.pdf

21/06/2020 21:08 137 178 Preguntas EDI.pdf

22/06/2020 04:20 165 878 Privacidad diferencial.pdf

8 archivos 1 421 843 bytes

2 dirs 40 350 986 240 bytes libres

C:\Users\lgesquivel\Desktop\2020-I\BDA\CompartirConocimiento>



Google File System

¿Qué hace un sistema de archivos?

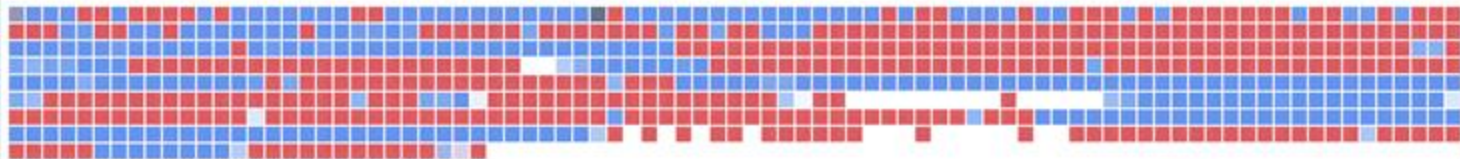


1. Divide un archivo en bloques de almacenamiento
 - Guarda la ubicación y secuencia de los bloques de un archivo



Drive	Name	Action	Status	Total files	Frag. files	Degree of fragmentation	Size	Free	File system	Current file/folder	Re...
	C: System	Ready	0%	232.771	379	0,67%	194,96 GB	104,42 GB	NTFS		
	D:	Ready	0%	41	0	0,00%	10,00 GB	9,95 GB	NTFS		
	E: Data II	Ready	0%	121.351	8.398	52,06%	172,78 GB	108,62 GB	NTFS		

C: 24.969 Cluster/Block

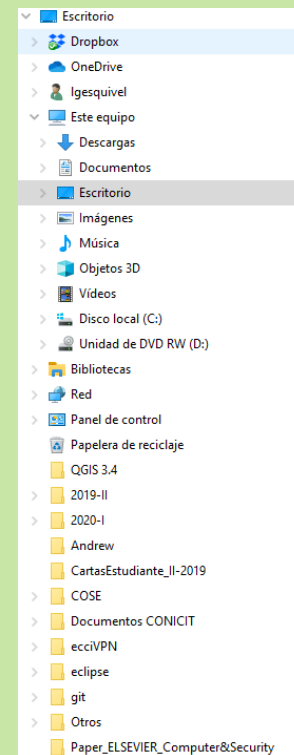


Google File System

¿Qué hace un sistema de archivos?



1. Divide un archivo en bloques de almacenamiento
 - Guarda la ubicación y secuencia de los bloques de un archivo
2. Organiza una estructura jerárquica de carpetas
 - Guarda las carpetas y los archivos en una carpeta



Google File System

¿Qué hace un sistema de archivos?



1. Divide un archivo en bloques de almacenamiento
 - Guarda la ubicación y secuencia de los bloques de un archivo
2. Organiza una estructura jerárquica de carpetas
 - Guarda las carpetas y los archivos en una carpeta
3. Guarda los meta datos de los archivos
 - Tamaño de los archivos, fecha de creación



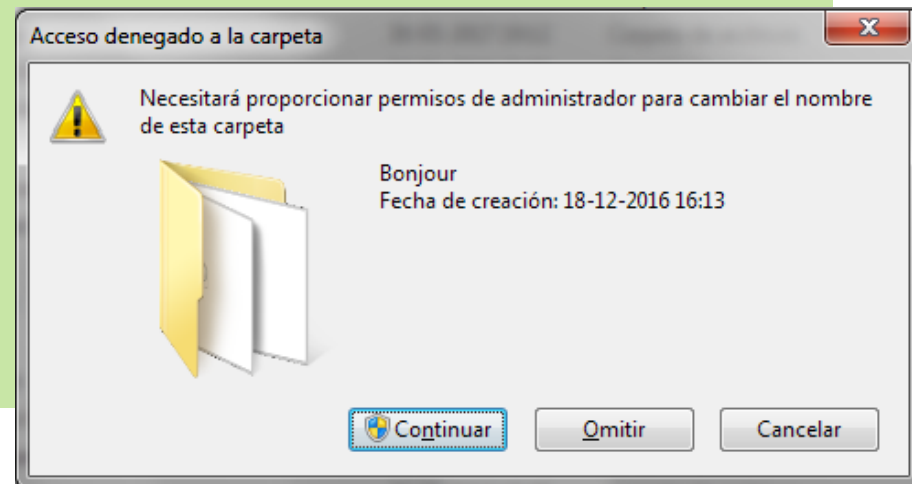
Nombre	Fecha de modificación	Tipo	Tamaño	Fecha de creación
 CAP_Base	12/5/2020 08:48	Carpeta de archivos		12/5/2020 08:48
 ConceptosDistribuidos	12/5/2020 08:48	Carpeta de archivos		12/5/2020 08:48

Google File System

¿Qué hace un sistema de archivos?



1. Divide un archivo en bloques de almacenamiento
 - Guarda la ubicación y secuencia de los bloques de un archivo
2. Organiza una estructura jerárquica de carpetas
 - Guarda las carpetas y los archivos en una carpeta
3. Guarda los meta datos de los archivos
 - Tamaño de los archivos, fecha de creación
4. Protege los archivos
 - Propiedad, permisos, bloqueos de sincronización

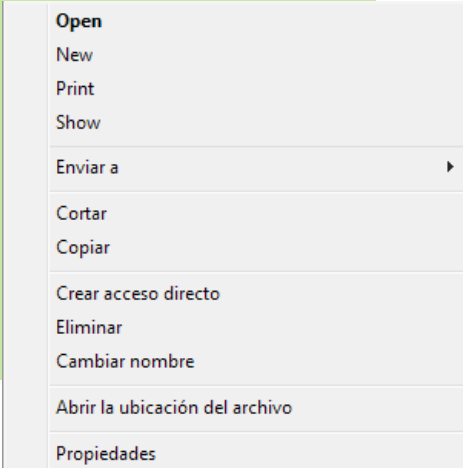


Google File System

¿Qué hace un sistema de archivos?



1. Divide un archivo en bloques de almacenamiento
 - Guarda la ubicación y secuencia de los bloques de un archivo
2. Organiza una estructura jerárquica de carpetas
 - Guarda las carpetas y los archivos en una carpeta
3. Guarda los meta datos de los archivos
 - Tamaño de los archivos, fecha de creación
4. Protege los archivos
 - Propiedad, permisos, bloqueos de sincronización
5. Provee una interfaz para interactuar con archivos
 - Crear, borrar, copiar, etc.



A vertical list of file management actions, likely a context menu. The items are: Open, New, Print, Show, Enviar a (with a right-pointing arrow), Cortar, Copiar, Crear acceso directo, Eliminar, Cambiar nombre, Abrir la ubicación del archivo, and Propiedades.

- Open
- New
- Print
- Show
- Enviar a
- Cortar
- Copiar
- Crear acceso directo
- Eliminar
- Cambiar nombre
- Abrir la ubicación del archivo
- Propiedades

Google File System

¿Qué hace el "Google File System"?



The Google File System

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung

Google*

ABSTRACT

We have designed and implemented the Google File System, a scalable distributed file system for large distributed data-intensive applications. It provides fault tolerance while running on inexpensive commodity hardware, and it delivers high aggregate performance to a large number of clients.

While sharing many of the same goals as previous distributed file systems, our design has been driven by observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure from some earlier file system assumptions. This has led us to reexamine traditional choices and explore radically different design points.

The file system has successfully met our storage needs. It is widely deployed within Google as the storage platform for the generation and processing of data used by our service as well as research and development efforts that require large data sets. The largest cluster to date provides hundreds of terabytes of storage across thousands of disks on over a thousand machines, and it is concurrently accessed by hundreds of clients.

In this paper, we present file system interface extensions designed to support distributed applications, discuss many aspects of our design, and report measurements from both micro-benchmarks and real world use.

1. INTRODUCTION

We have designed and implemented the Google File System (GFS) to meet the rapidly growing demands of Google's data processing needs. GFS shares many of the same goals as previous distributed file systems such as performance, scalability, reliability, and availability. However, its design has been driven by key observations of our application workloads and technological environment, both current and anticipated, that reflect a marked departure from some earlier file system design assumptions. We have reexamined traditional choices and explored radically different points in the design space.

First, component failures are the norm rather than the exception. The file system consists of hundreds or even thousands of storage machines built from inexpensive commodity parts and is accessed by a comparable number of client machines. The quantity and quality of the components virtually guarantee that some are not functional at any given time and some will not recover from their current failures. We have seen problems caused by application bugs, operating system bugs, human errors, and the failures of disks, memory, connectors, networking, and power supplies. Therefore, constant monitoring, error detection, fault tolerance, and automatic recovery must be integral to the system.

Google File System

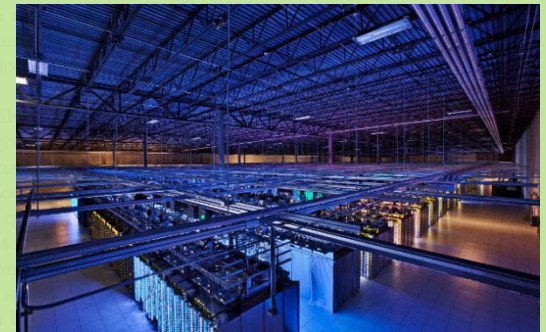
¿Qué hace el "Google File System"?



1. Divide un archivo en bloques de almacenamiento
 - Guarda la ubicación y secuencia de los bloques de un archivo
2. Organiza una estructura jerárquica de carpetas
 - Guarda las carpetas y los archivos en una carpeta
3. Guarda los meta datos de los archivos
 - Tamaño de los archivos, fecha de creación
4. Protege los archivos
 - Propiedad, permisos, bloqueos de sincronización
5. Proviene una interfaz para interactuar con archivos
 - Crear, borrar, copiar, etc.



**Lo mismo, pero distribuido:
(y abstrae la distribución)**



In this paper, we present file system interface extensions designed to support distributed applications, discuss many aspects of our design, and report measurements from both micro-benchmarks and real world use.

of disks, memory, connectors, networking, and power supplies. Therefore, constant monitoring, error detection, fault tolerance, and automatic recovery must be integral to the system.

Google File System: Suposiciones

- Archivos enormes
- Archivos frecuentemente leídos o anexados
- Concurrencia es importante
- Fallas son frecuentes
- "Streaming" es importante



GFS: Arquitectura

- 64 MB por bloque
- Etiqueta de 64 bit para cada bloque
- Supone factor de replicación: 3

Sistema de archivos (en memoria)

Maestro



A1

B1

C1

D1

E1



Esclavos

GFS: Escritura

- 64 MB por bloque
- Etiqueta de 64 bit para cada bloque
- Sponga factor de replicación: 3

Sistema de archivos (en memoria)

Maestro



`blue.txt`

(150 MB: 3 bloques)



¿Cómo deberíamos proceder con la escritura?



A1

B1

C1

D1

E1



Esclavos

GFS: Escritura

- 64 MB por bloque
- Etiqueta de 64 bit para cada bloque
- Suponer factor de replicación: 3

Sistema de archivos (en memoria)

/blue.txt [3 bloques]

1: {A1, C1, E1}

2: {A1, B1, D1}

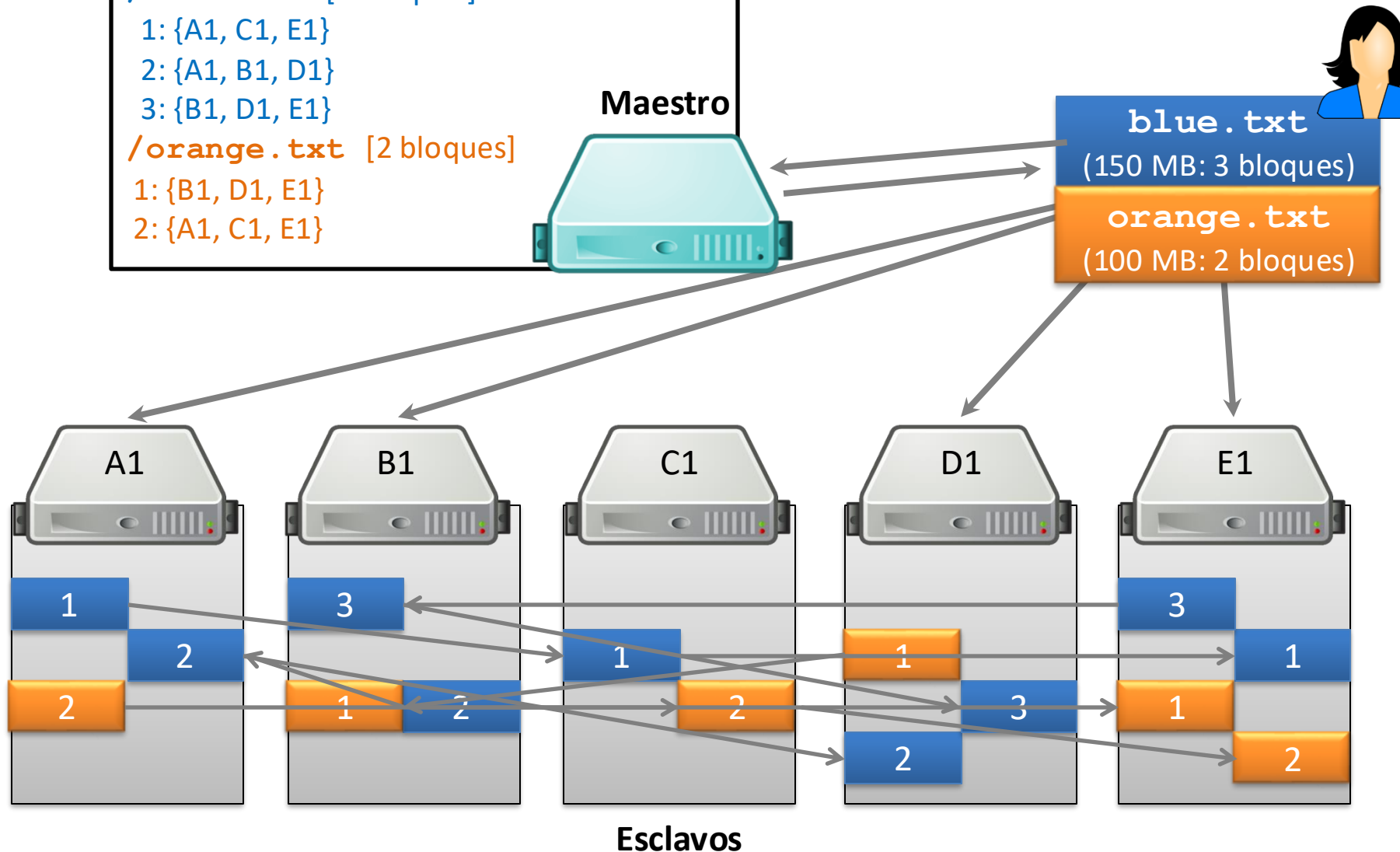
3: {B1, D1, E1}

/orange.txt [2 bloques]

1: {B1, D1, E1}

2: {A1, C1, E1}

Maestro



GFS: Tolerancia a fallos

- 64 MB por bloque
- Etiqueta de 64 bit para cada bloque
- Suponer factor de replicación: 3

Sistema de archivos (en memoria)

/blue.txt [3 bloques]

1: {A1, B1, E1}

2: {A1, B1, D1}

3: {B1, D1, E1}

/orange.txt [2 bloques]

1: {B1, D1, E1}

2: {A1, D1, E1}

Maestro

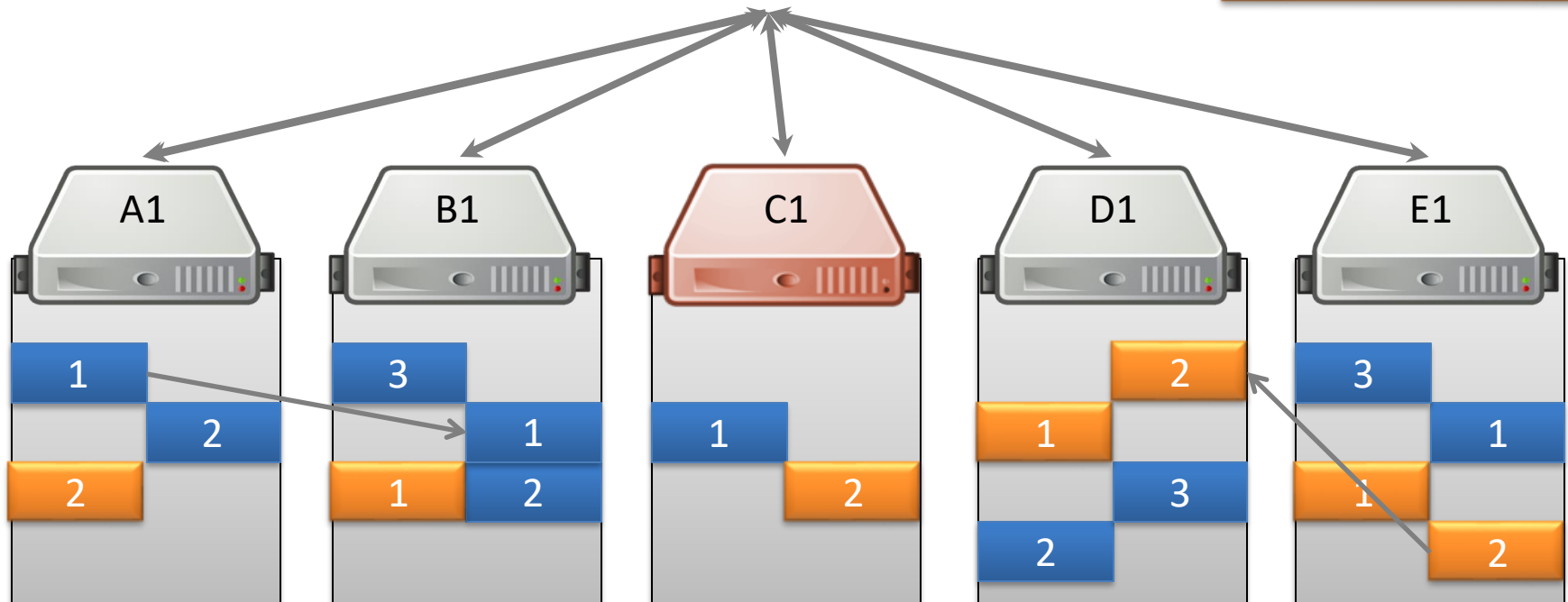


blue.txt

(150 MB: 3 bloques)

orange.txt

(100 MB: 2 bloques)



Esclavos

GFS: Lecturas directas

- 64 MB por bloque
- Etiqueta de 64 bit
- Suponer factor de

Sistema de archivos (en memoria)

/blue.txt [3 bloques]

1: {A1, C1, E1}

2: {A1, B1, D1}

3: {B1, D1, E1}

/orange.txt [2 bloques]

1: {B1, D1, E1}

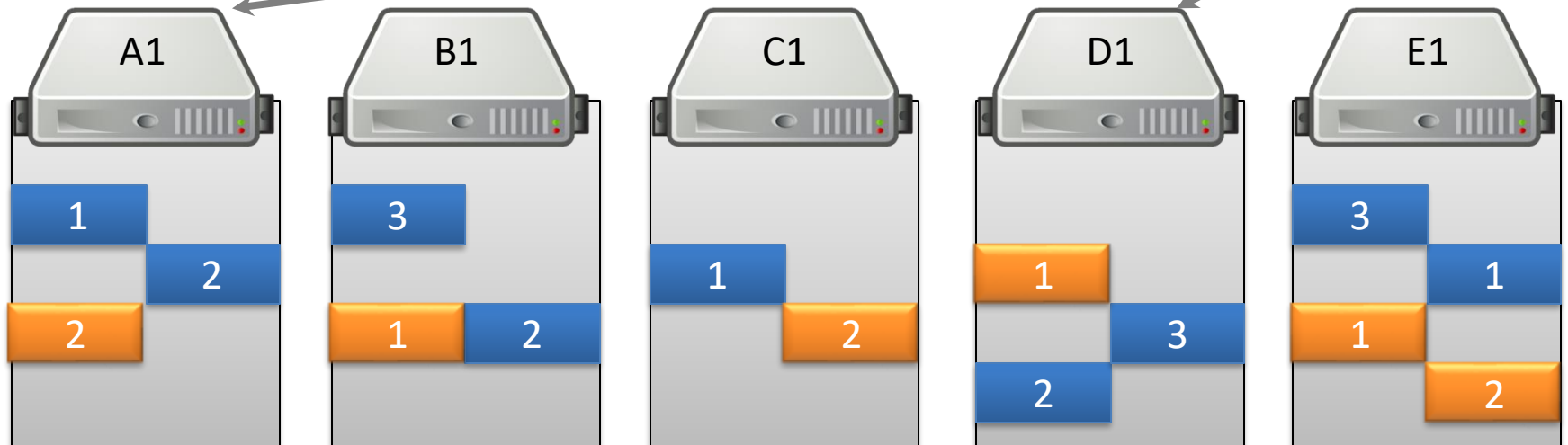
2: {A1, C1, E1}

Maestro

Busco
/blue.txt

1 2 3

/blue.txt [3 chunks]
1: {A1, C1, E1}
2: {A1, B1, D1}
3: {B1, D1, E1}



Esclavos

GFS: Replicas Primarias

- 64 MB por bloque
- Etiqueta de 64 bit
- Suponer factor de

Sistema de archivos (en memoria)

/blue.txt [3 bloques]

1: {A1, C1, E1}

2: {A1, B1, D1}

3: {B1, D1, E1}

/orange.txt [2 bloques]

1: {B1, D1, E1}

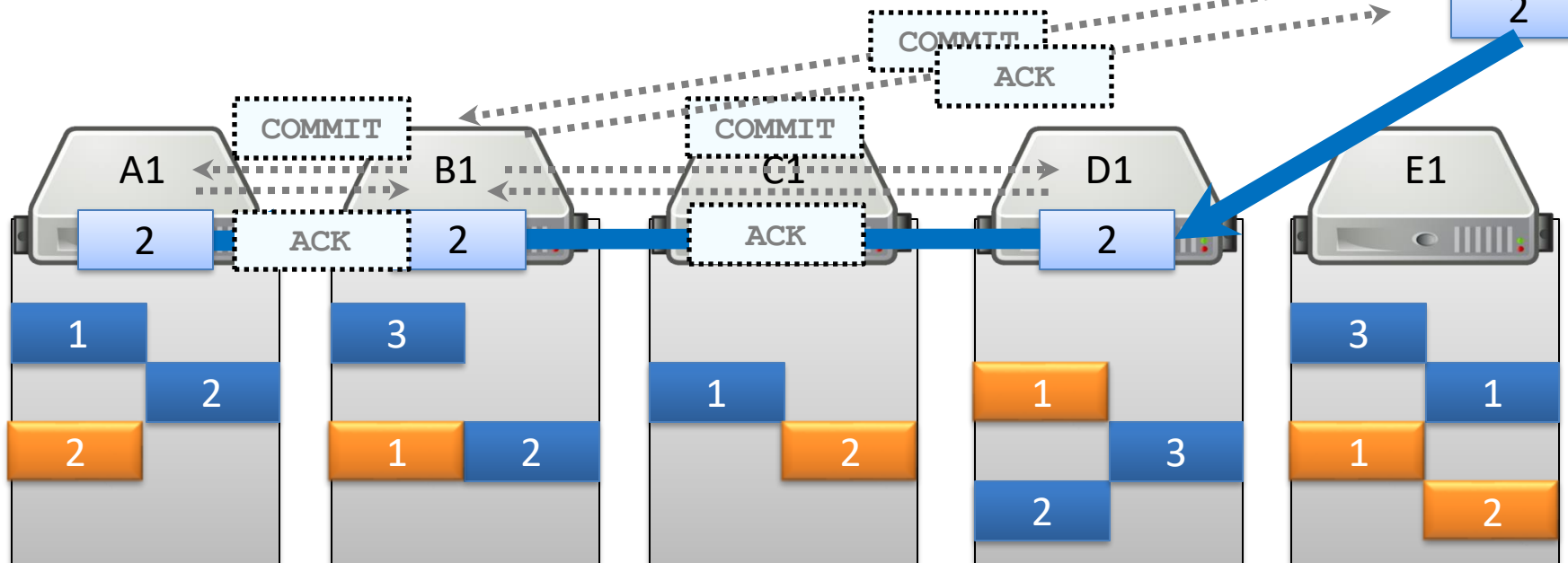
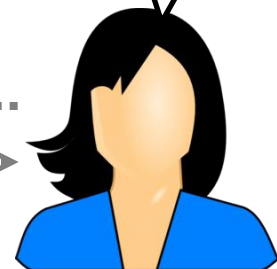
2: {A1, C1, E1}

Maestro



/blue.txt [3 chunks]
2: {A1, B1, D1}

Quiero cambiar
el bloque 2 de
/blue.txt



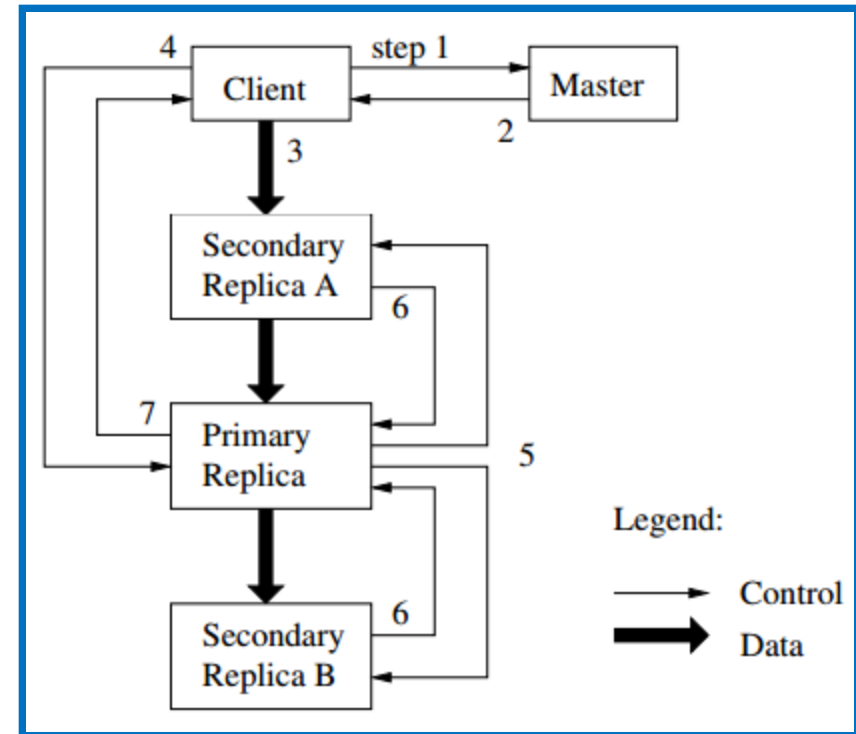
Esclavos

GFS: Replicas Primarias

Maestro asigna tarea a una de las réplicas: “réplica primaria”

Cliente quiere modificar un archivo:

1. Cliente solicita réplicas (incl. la primaria) al Maestro
2. Maestro retorna información de la réplica al cliente
3. Cliente envía cambios del archivo
4. Cliente solicita ejecutar los cambios a la réplica primaria
5. Primaria solicita ejecutar los cambios a secundarias
6. Secundarias notifican a primaria
7. Primaria notifica al cliente

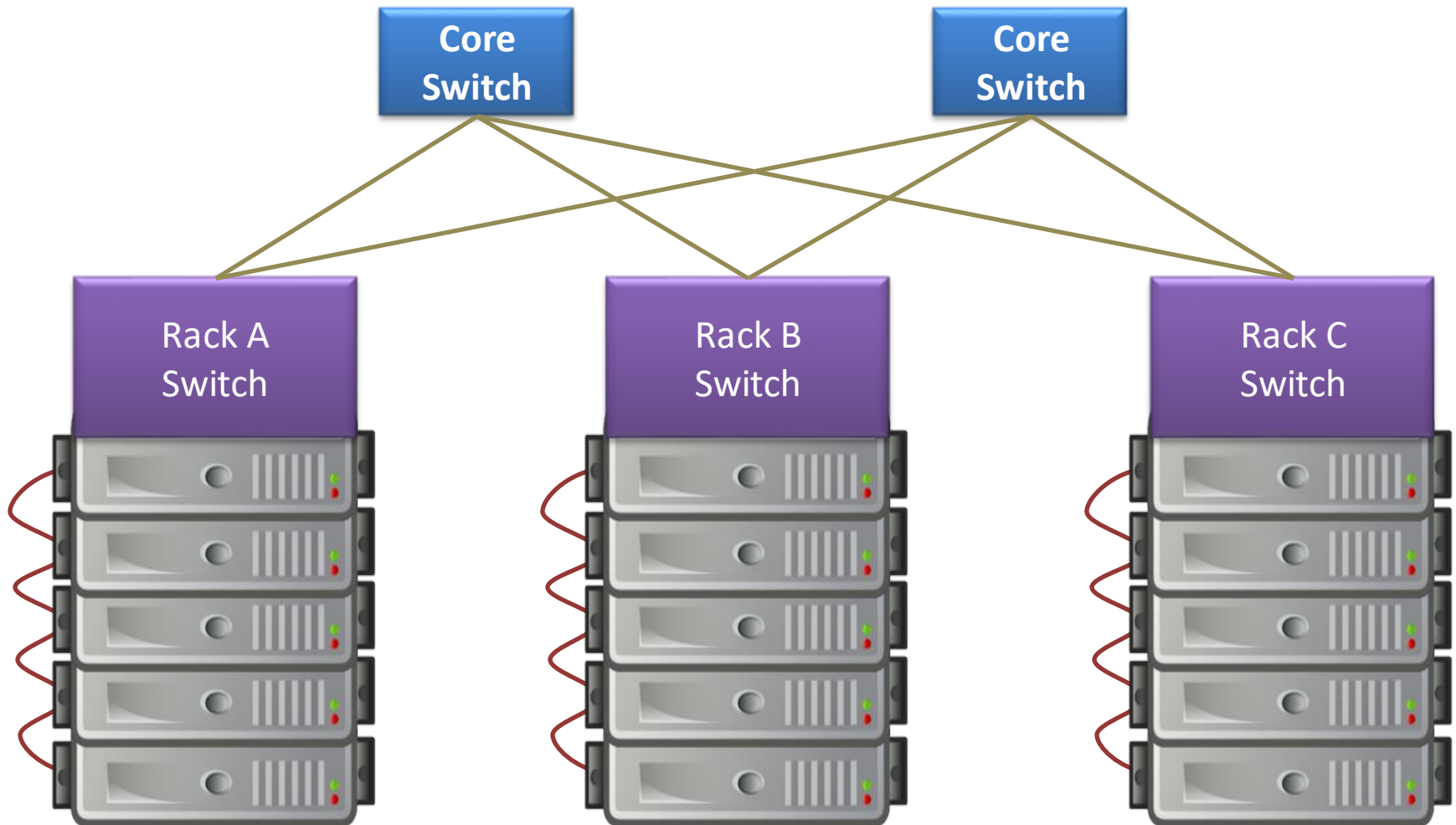


Datos y Control desacoplados

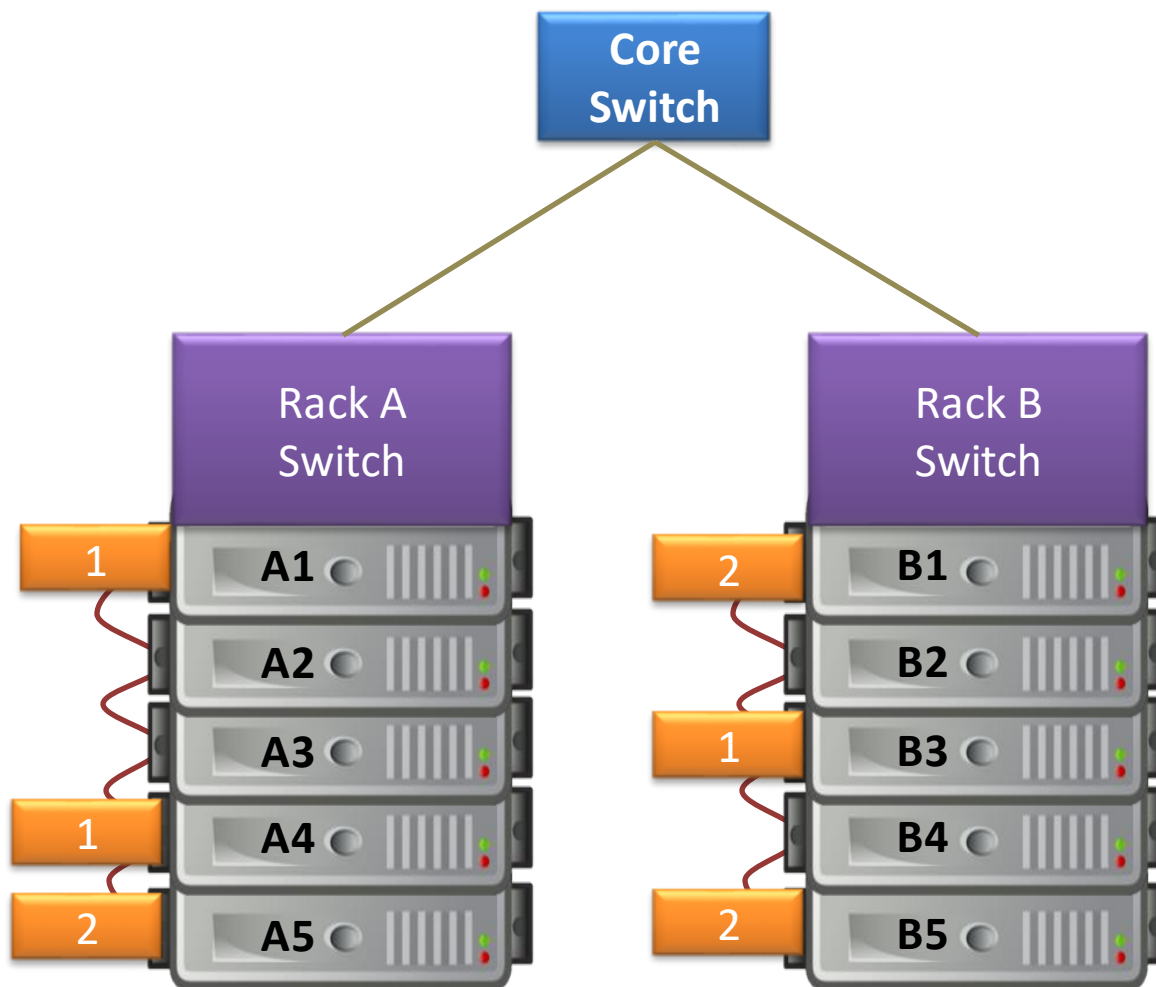
GFS: Conocimiento sobre Racks



GFS: Conocimiento sobre Racks



GFS: Conocimiento sobre Racks



Archivos:

`/orange.txt`

1: {A1, A4, B3}

2: {A5, B1, B5}

Racks:

A: {A1, A2, A3, A4, A5}

B: {B1, B2, B3, B4, B5}

GFS: Otras operaciones

Rebalanceo:

- Distribuir el almacenamiento equitativamente

Borrado:

- Renombrar el archivo con nombre de archivo oculto
 - Para recuperar, volver a renombrar a versión original
 - Si no, será eliminado después de tres días

Monitoreo de réplicas obsoletas:

- Esclavo “muerto” reaparece con datos antiguos: maestro guardará información de la versión y reciclará bloques viejos

GFS: ¿Debilidades?

¿Cuales son las debilidades principales del GFS?



Maestro es un único punto de falla



- Usar replicación de hardware
- ¡Logs y checkpoints!

Maestro es un cuello de botella



- Usar una máquina más poderosa
- Minimizar tráfico por nodo maestro

Meta-datos del nodo maestro se mantiene en memoria



- Cada bloque sólo necesita 64 bytes en memoria
- Datos del bloque pueden ser consultados en cada esclavo

Hadoop Distributed File System (HDFS)

- Versión open-source de GFS
 - Esclavo = "Data node"
 - Maestro = "Name node"



<https://hadoop.apache.org/>

Sistema de archivos distribuido ...

Crawling

1. Parsear enlaces de las páginas
2. Ordenar los enlaces para bajar
3. Bajar páginas, GOTO 1

Indexación

1. Parsear keywords de las páginas
2. Indexear sus keywords
3. Administrar actualizaciones

Ranking

1. ¿Cuán relevante es una página?
2. ¿Cuán importante es?
3. ¿Cuántos clics tiene?

...

- `write(file f)` **HDFS/GFS**
- `read(file f)`
- `delete(file f)`
- `append(file f, data d)`

¿Eso no más?





LIMITACIONES DE ALMACENAMIENTO DISTRIBUIDO: TEOREMA CAP

Repasemos ACID

Para bases de datos tradicionales sin distribución ...

1. Atomicity:

- Atomicidad: Las transacciones son todo o nada

2. Consistency:

- Coherencia: No rompe las restricciones / reglas

3. Isolation:

- Aislamiento: Transacciones paralelas actúan como si fueran secuenciales

4. Durability

- Durabilidad: El sistema recuerda los cambios

¿Qué es CAP?

Tres garantías que un sistema distribuido podría ofrecer

1. Consistency:

- Coherencia: Todos los nodos tienen una visión coherente del sistema

2. Availability:

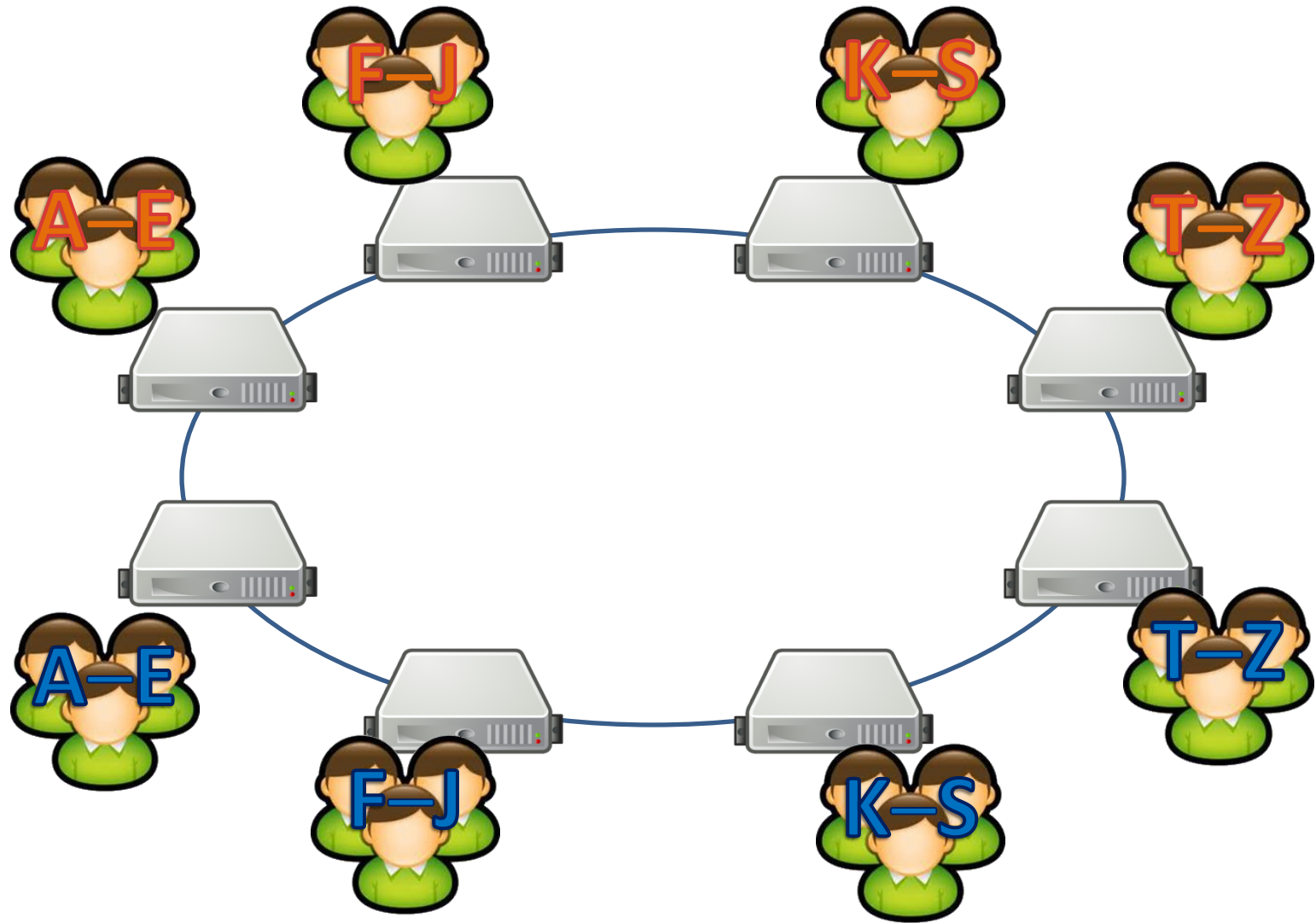
- Disponibilidad: Toda petición de escritura/lectura es atendida

3. Partition-tolerance:

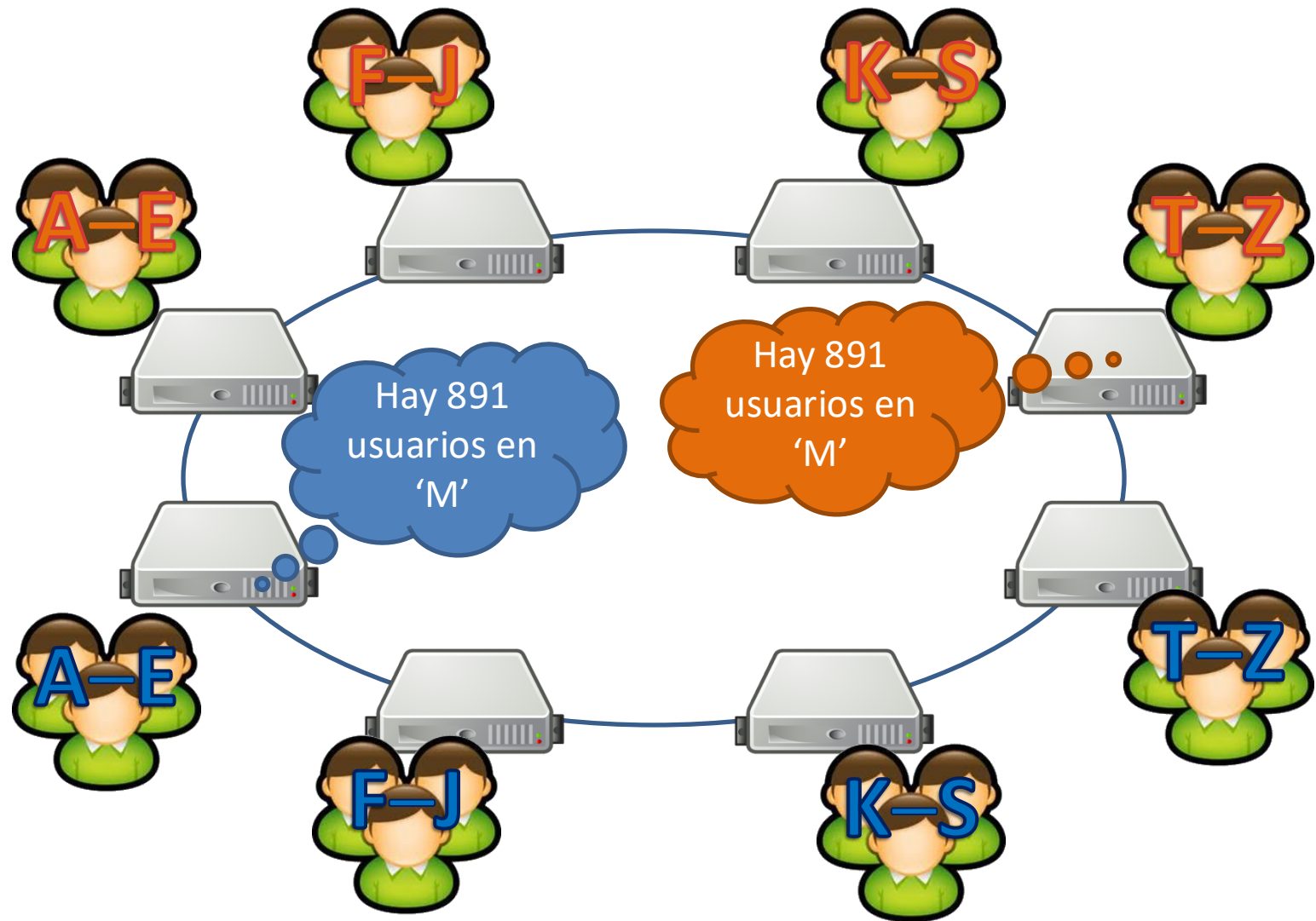
- Tolerancia a particiones: El sistema funciona incluso si hay pérdida de mensajes



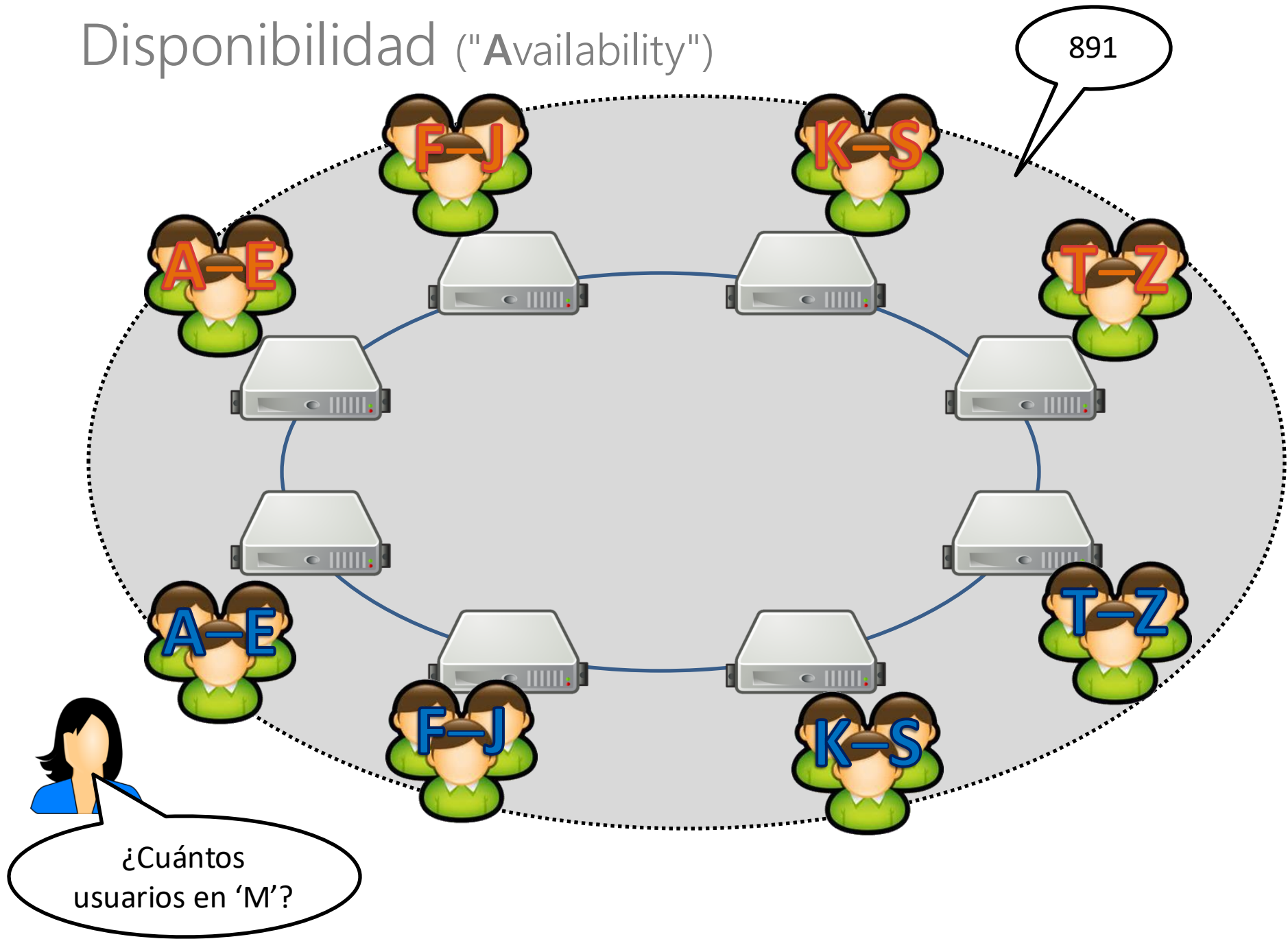
Un sistema distribuido (con Replicación)



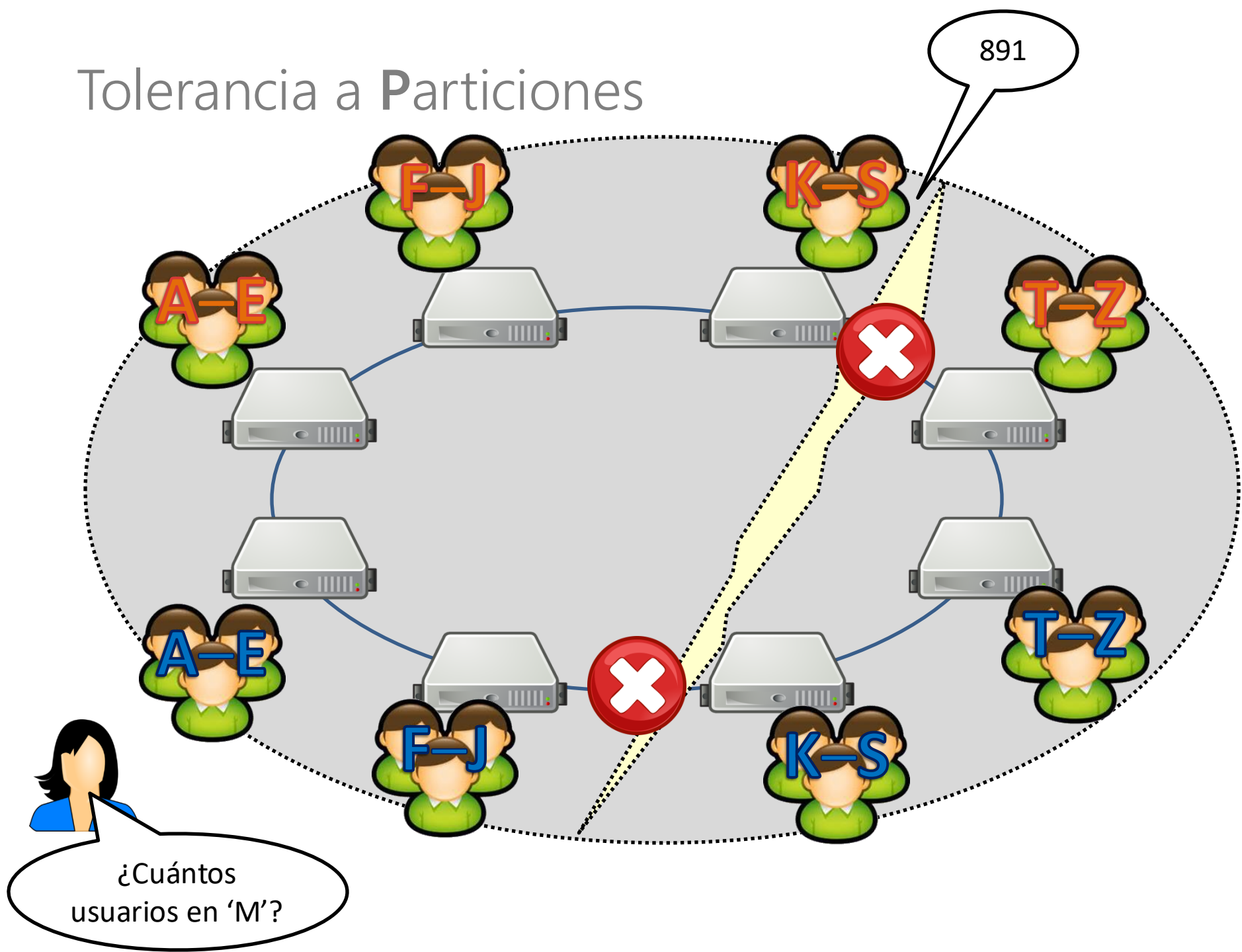
Coherencia



Disponibilidad ("Availability")



Tolerancia a Particiones



La pregunta CAP

¿Puede un sistema distribuido garantizar:

Coherencia (todos los nodos tienen una visión actualizada),

disponibilidad/Availability (toda escritura/lectura es atendida)

y

tolerancia a Particiones (el sistema funciona incluso si hay pérdida de mensajes)

al mismo tiempo?

¿Qué opinan?



La respuesta CAP



El teorema CAP

¡Un sistema distribuido no puede garantizar:

Coherencia (todos los nodos tienen una visión actualizada),

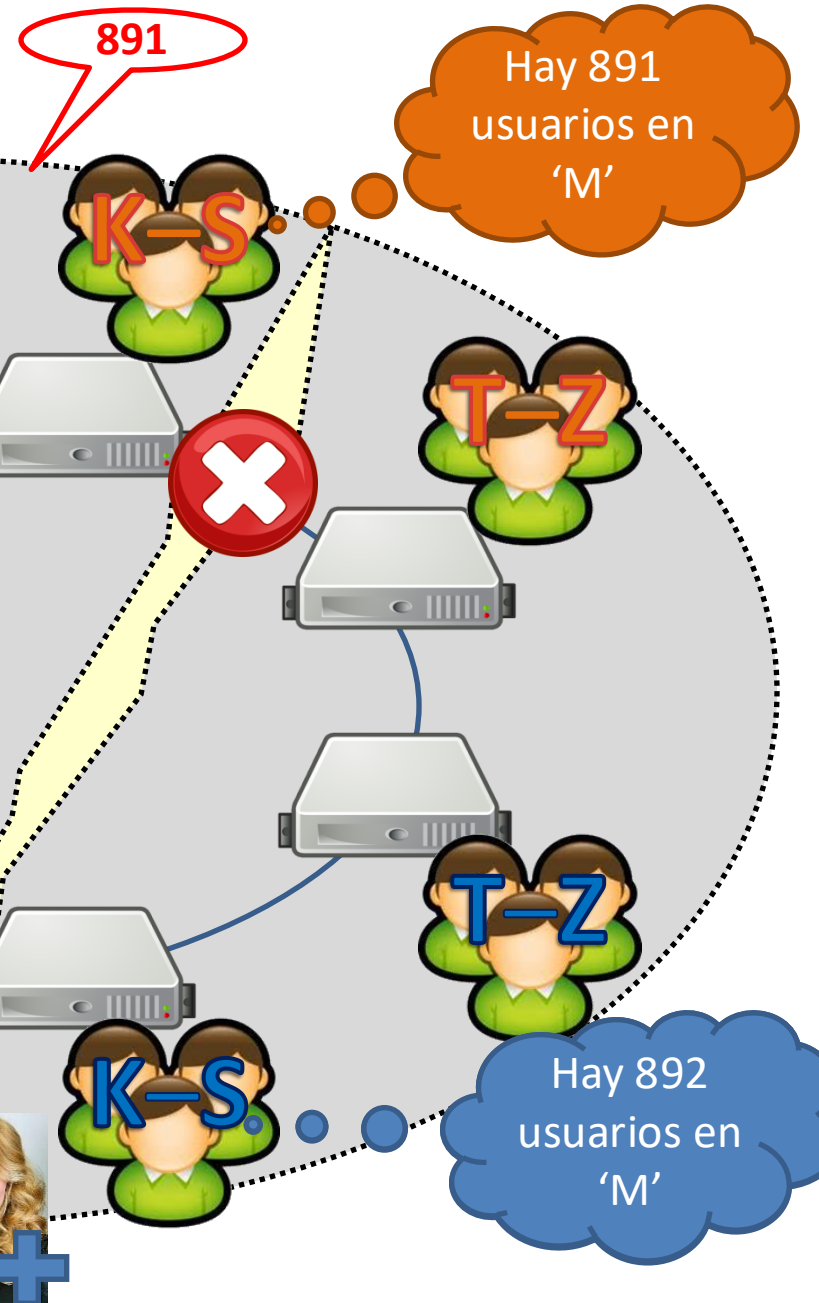
disponibilidad/Availability (toda escritura/lectura es atendida)

y

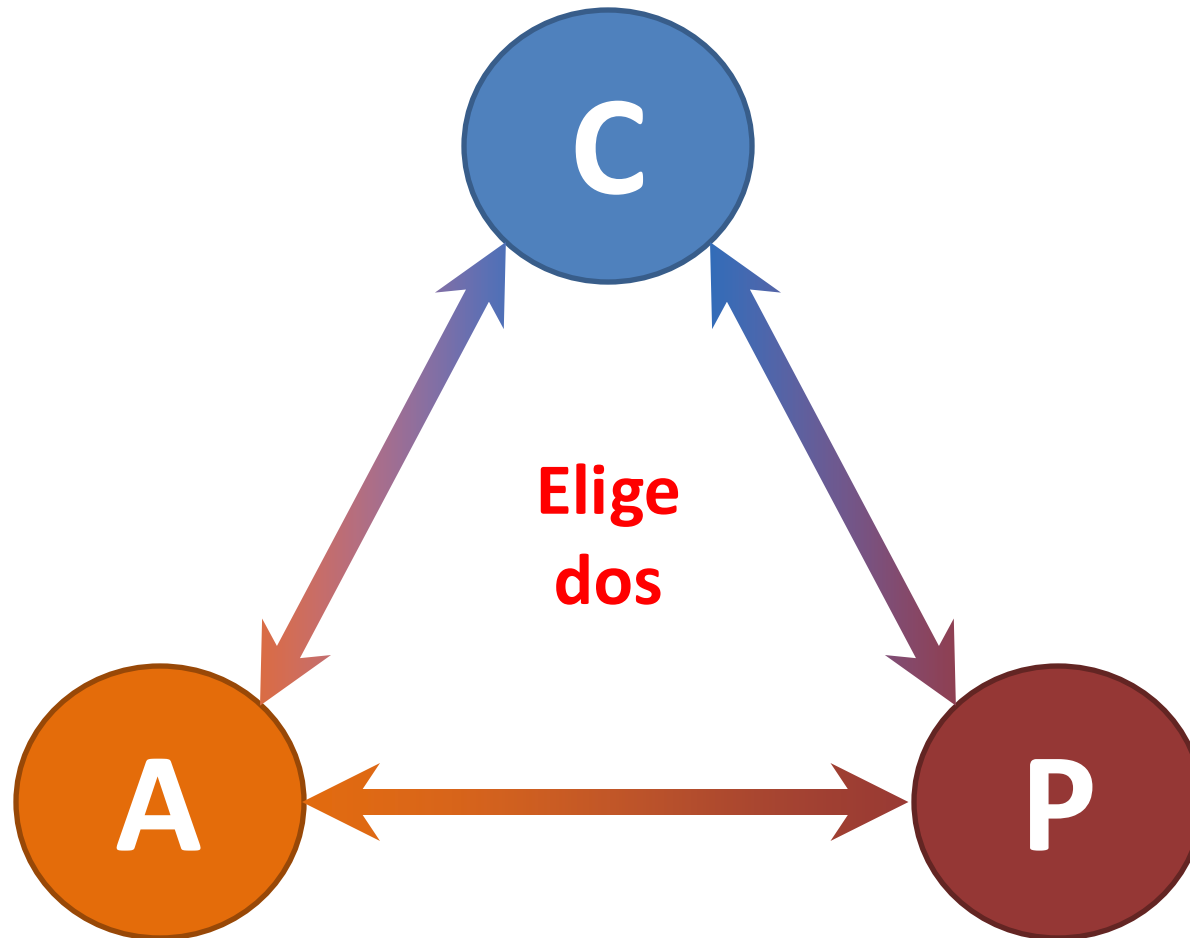
tolerancia a Particiones (el sistema funciona incluso si hay pérdida de mensajes)

al mismo tiempo!

La "demostración" CAP



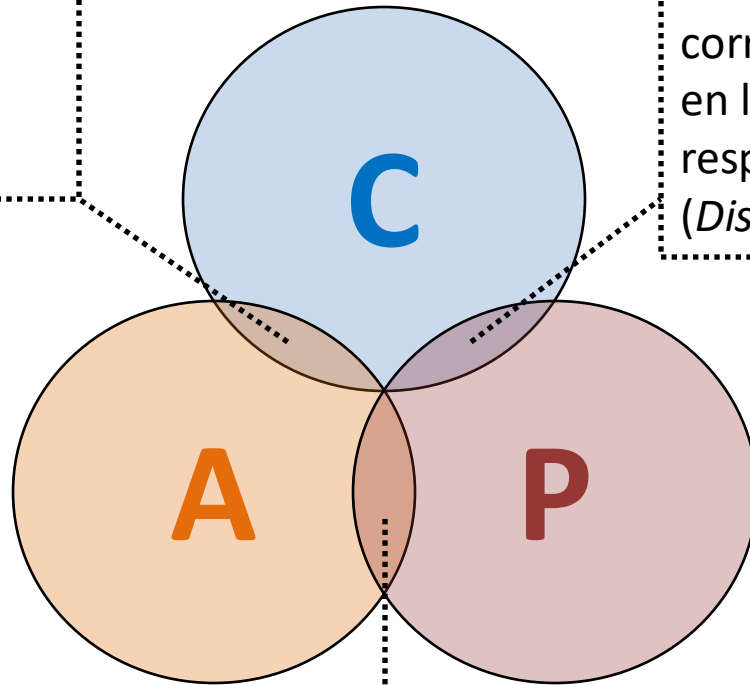
El triángulo CAP



Sistemas CAP

CA: Garantiza respuestas correctas sólo mientras la conexión funcione bien
(*Centralizado / Tradicional*)

CP: Garantiza respuestas correctas incluso si hay fallas en la conexión, pero la respuesta puede fracasar
(*Disponibilidad débil*)



(No hay intersección)

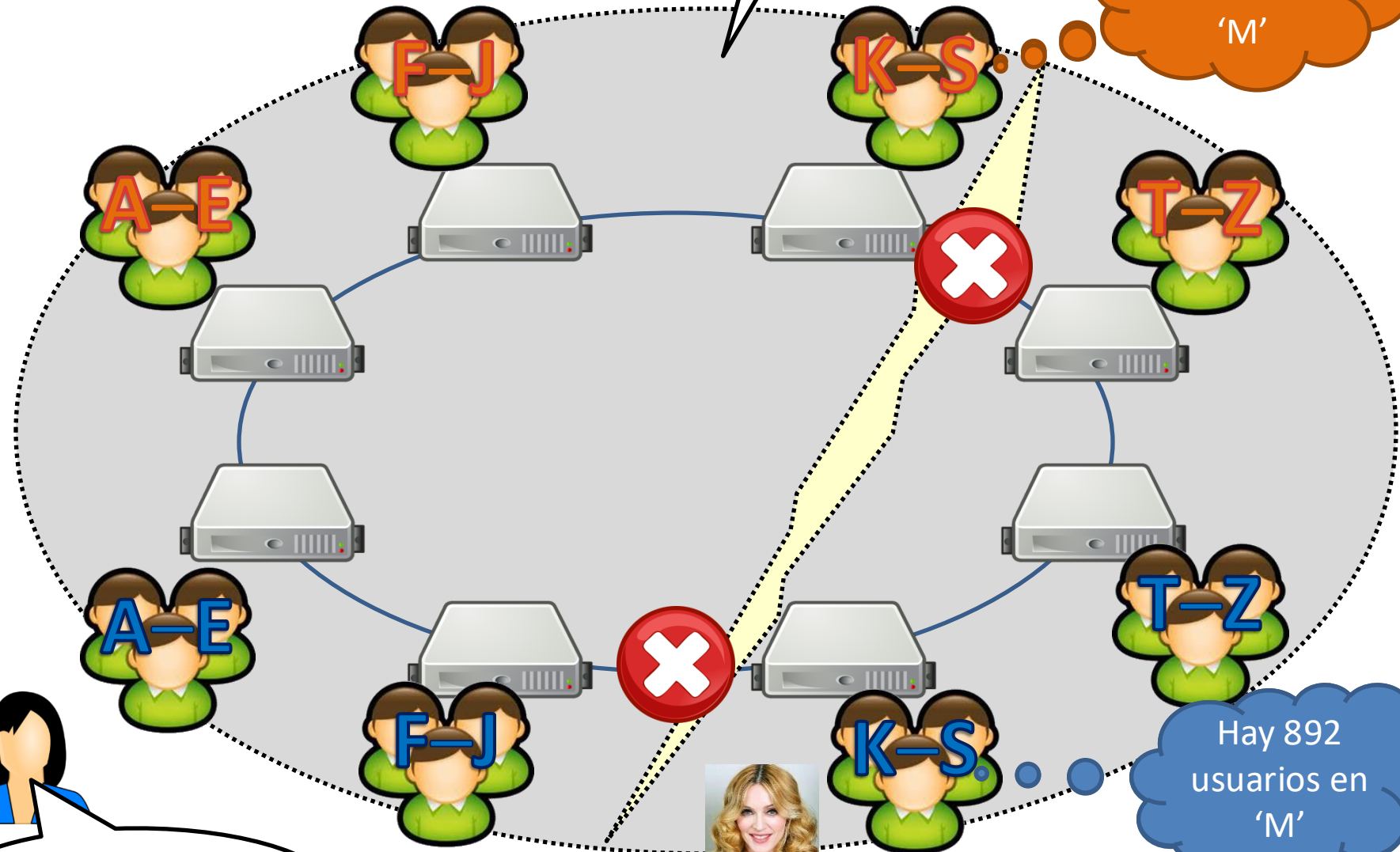
AP: Siempre provee "la mejor" respuesta que puede, esto incluso en presencia de fallas en la conexión
(*Coherencia eventual*)

Sistema CA

892

Hay 892
usuarios en
'M'

Hay 892
usuarios en
'M'

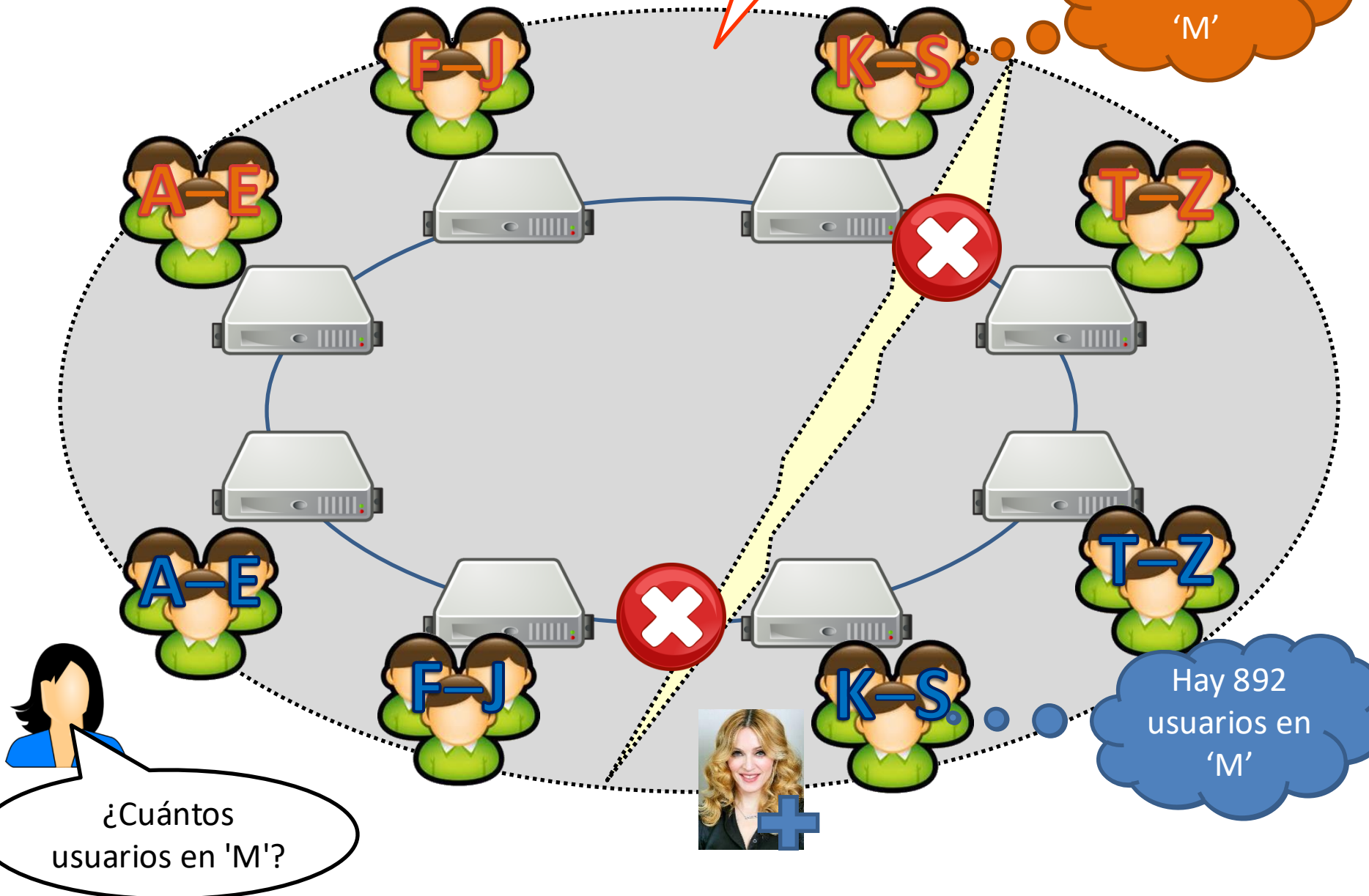


¿Cuántos
usuarios en 'M'?

Sistema CP

¡Error!

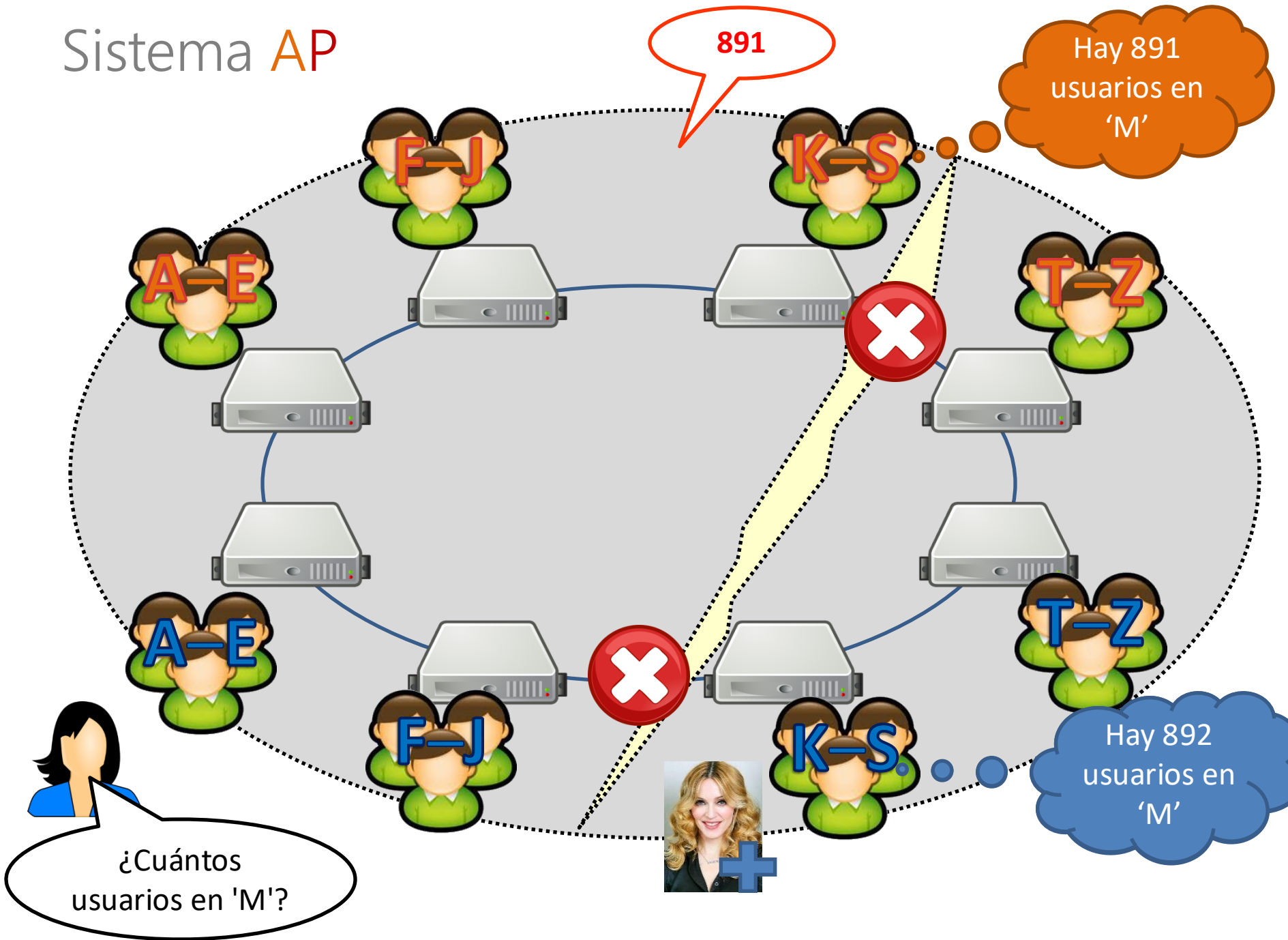
Hay 891
usuarios en
'M'



¿Cuántos
usuarios en 'M'?

Hay 892
usuarios en
'M'

Sistema AP



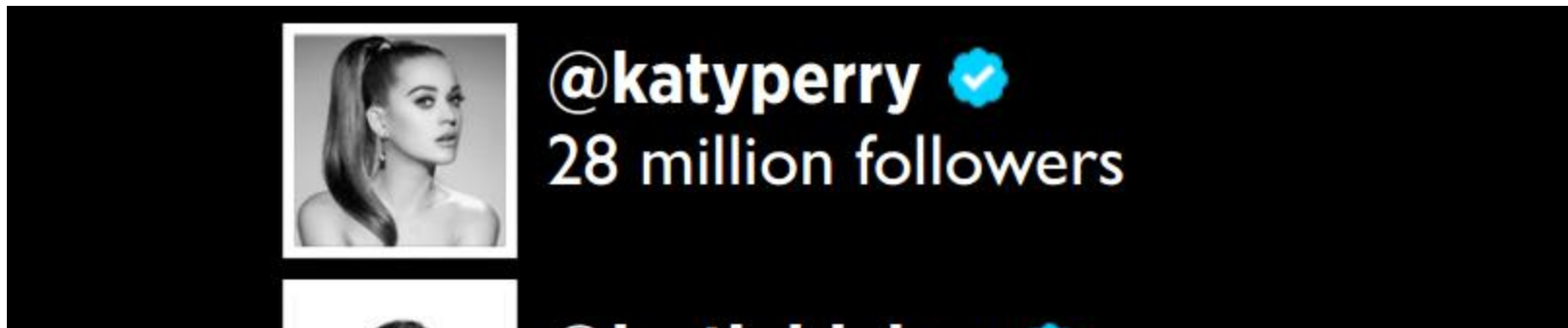
BASE (AP)

- **B**asically **A**vailable (básicamente disponible)
 - Casi siempre funciona (está operando)
- **S**oft State (estado virtualizado)
 - Información replicada / en caché
- **E**ventual Consistency (coherencia eventual)
 - Se toleran, por un tiempo, datos obsoletos

¿De qué manera Twitter opera como un sistema BASE (AP)?



Katy Perry crea una “partición” de la red

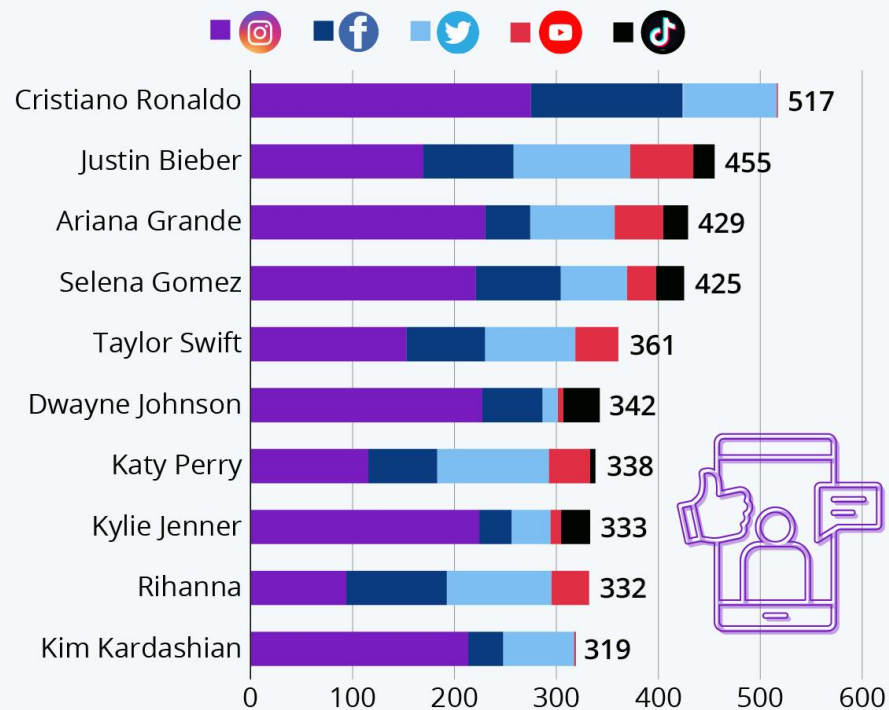


Los usuarios pueden ver los retweets
de los tweets de las celebridades
antes del tweet original.

Más tarde cuando llega el tweet original, el timeline
se ordenará de manera coherente.

Las personas más seguidas en redes

Personas con el mayor número de seguidores en redes sociales (en millones)*

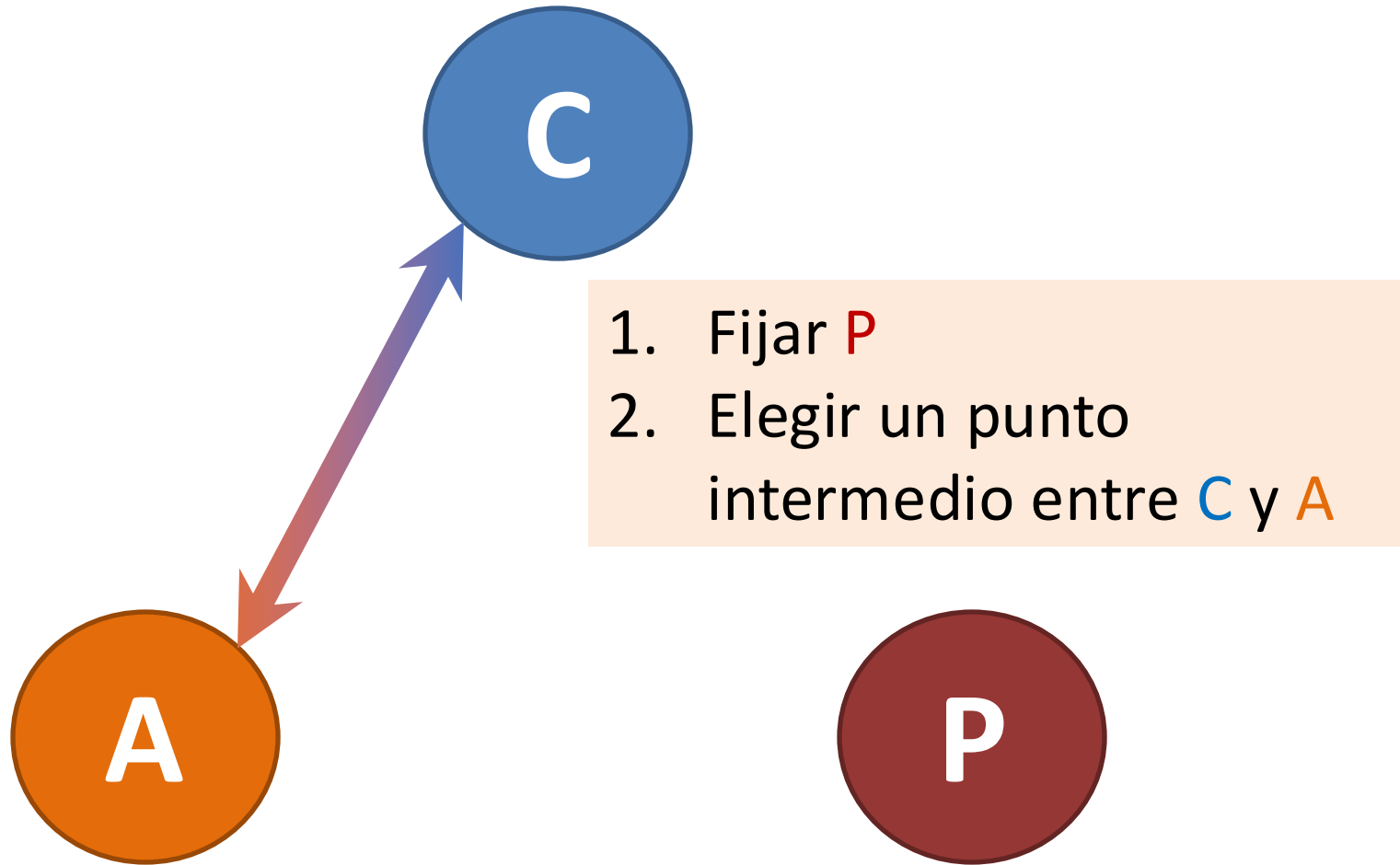


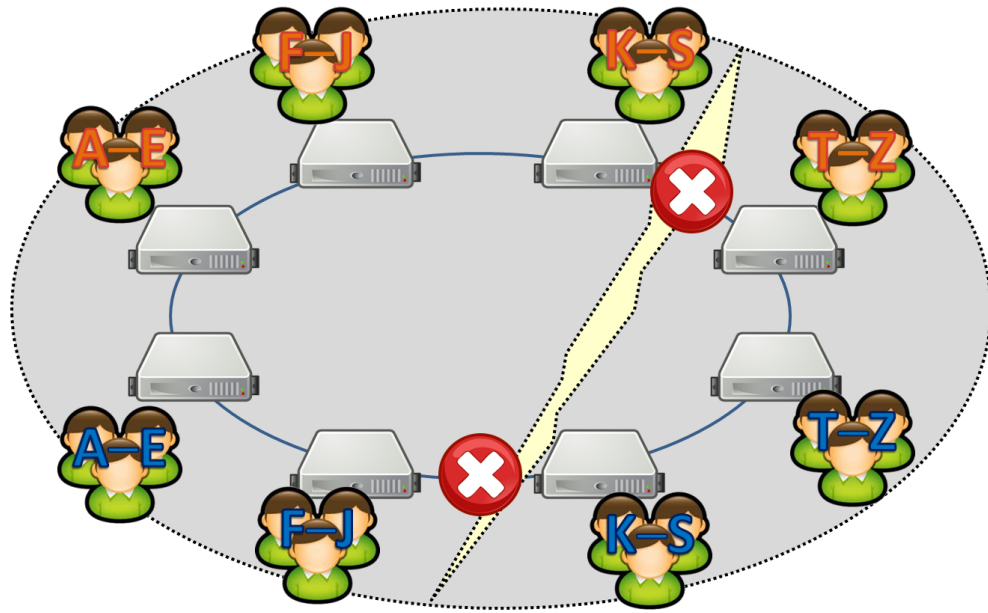
* Redes sociales seleccionadas. Datos de abril de 2021.

Fuente: Visual Capitalist



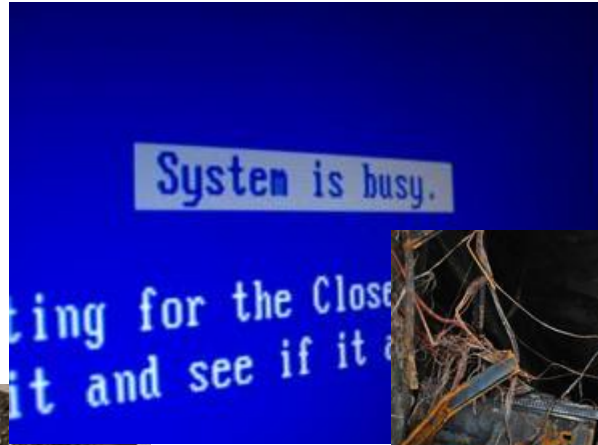
CAP en la práctica





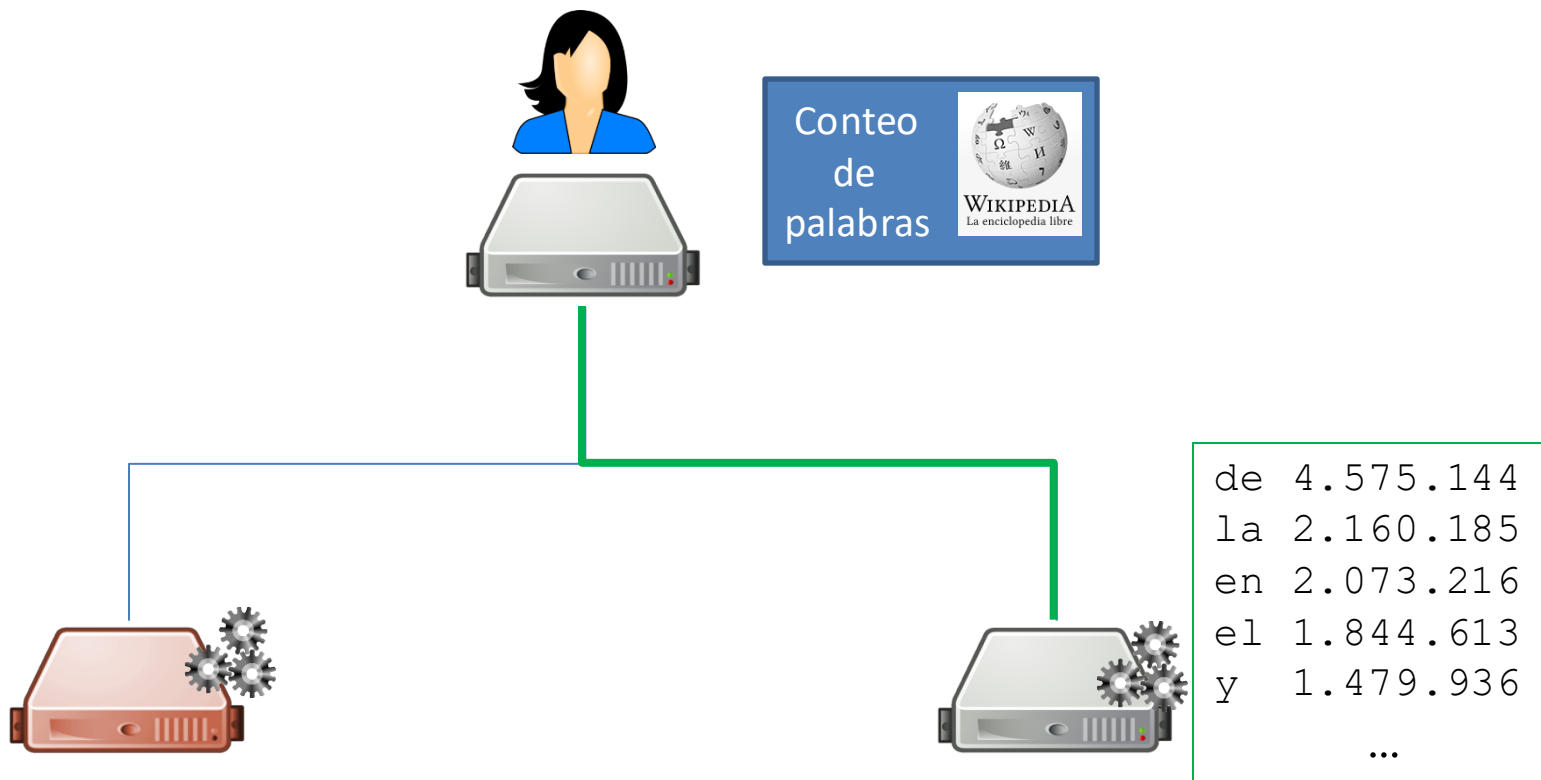
TOLERANCIA A PARTICIONES

Fallas



Falla "Fail-Stop"

- Una máquina no responde (a tiempo)
 - a menudo, por falla del hardware o por sobrecarga
 - se necesita al menos $f + 1$ máquinas replicadas
 - f = número de fallas "fail-stop"



Falla Bizantina

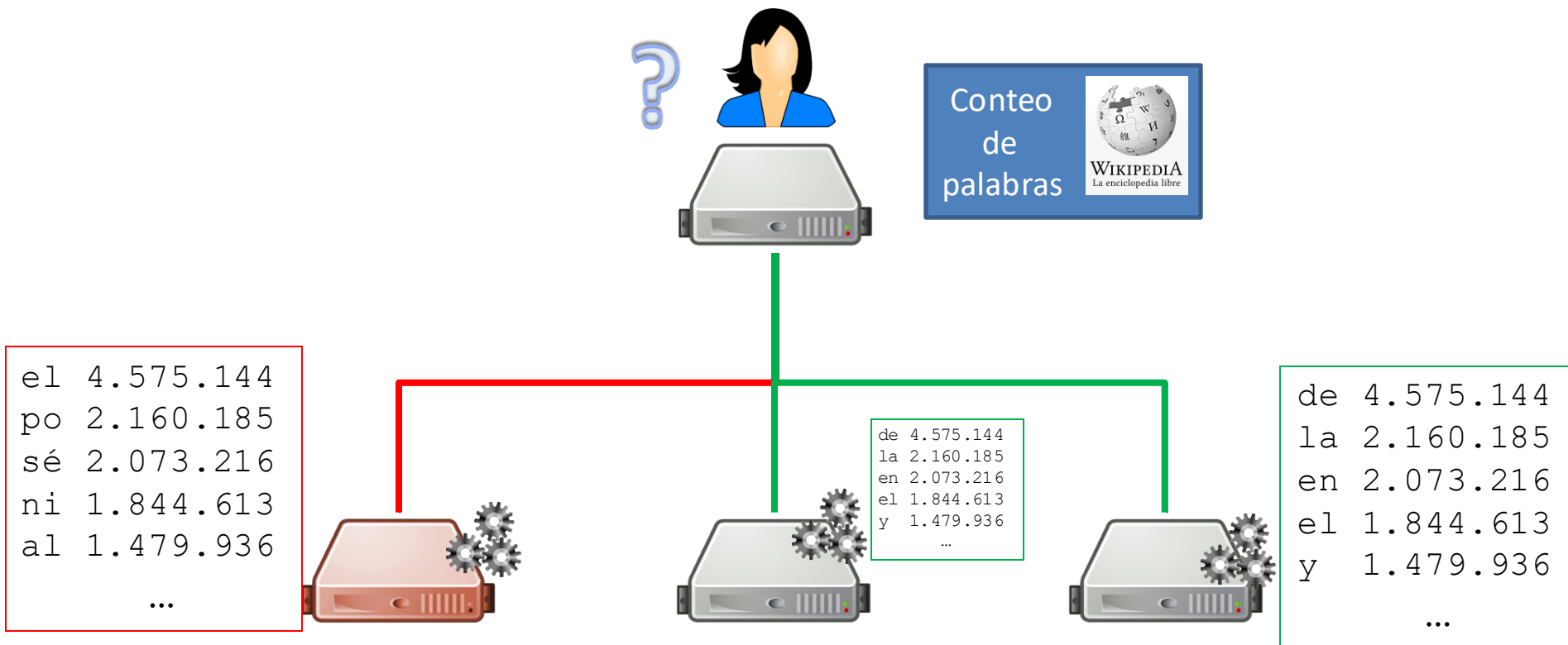


<https://youtu.be/dfsRQyYXOsQ>

Falla Bizantina

- Una máquina responde de forma incorrecta

¿Cuántas máquinas operativas necesitamos para corregir una falla bizantina?



Falla Bizantina

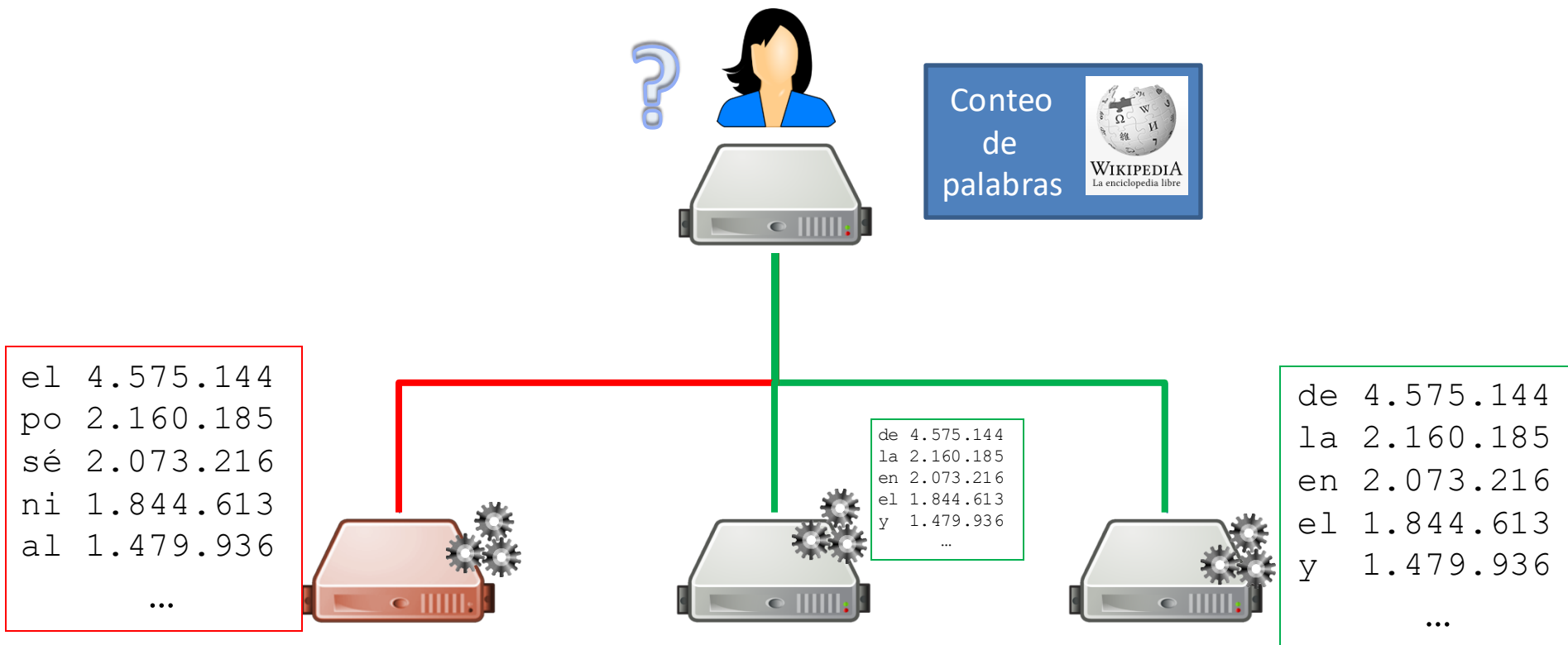
- Una máquina responde de forma incorrecta

¿Cuántas máquinas operativas necesitamos para corregir una falla bizantina?



Se necesitan $2f+1$ máquinas replicadas

- f = número de fallas (posiblemente) bizantinas



¿Consenso distribuido?

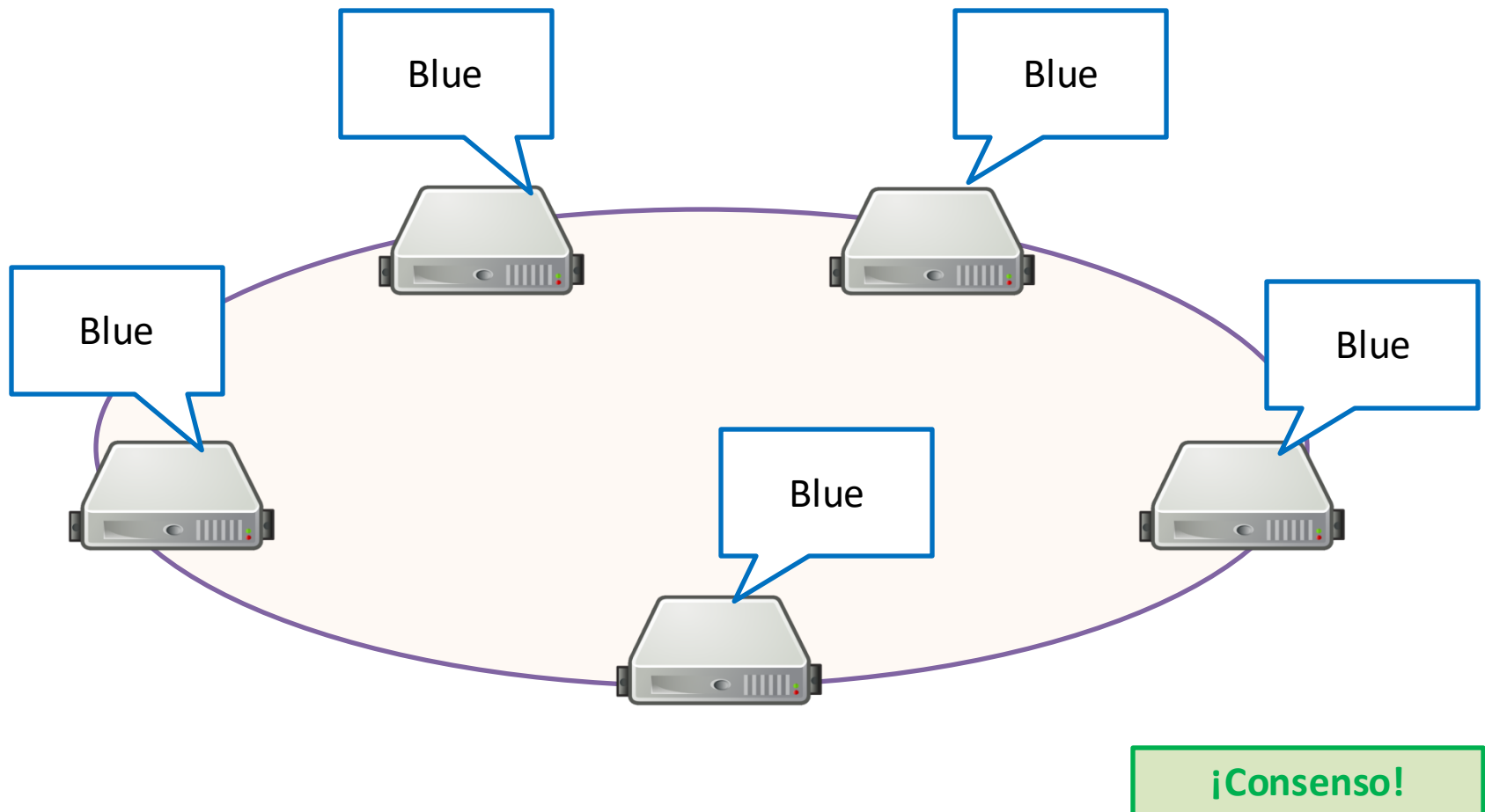


¿Color del vestido?



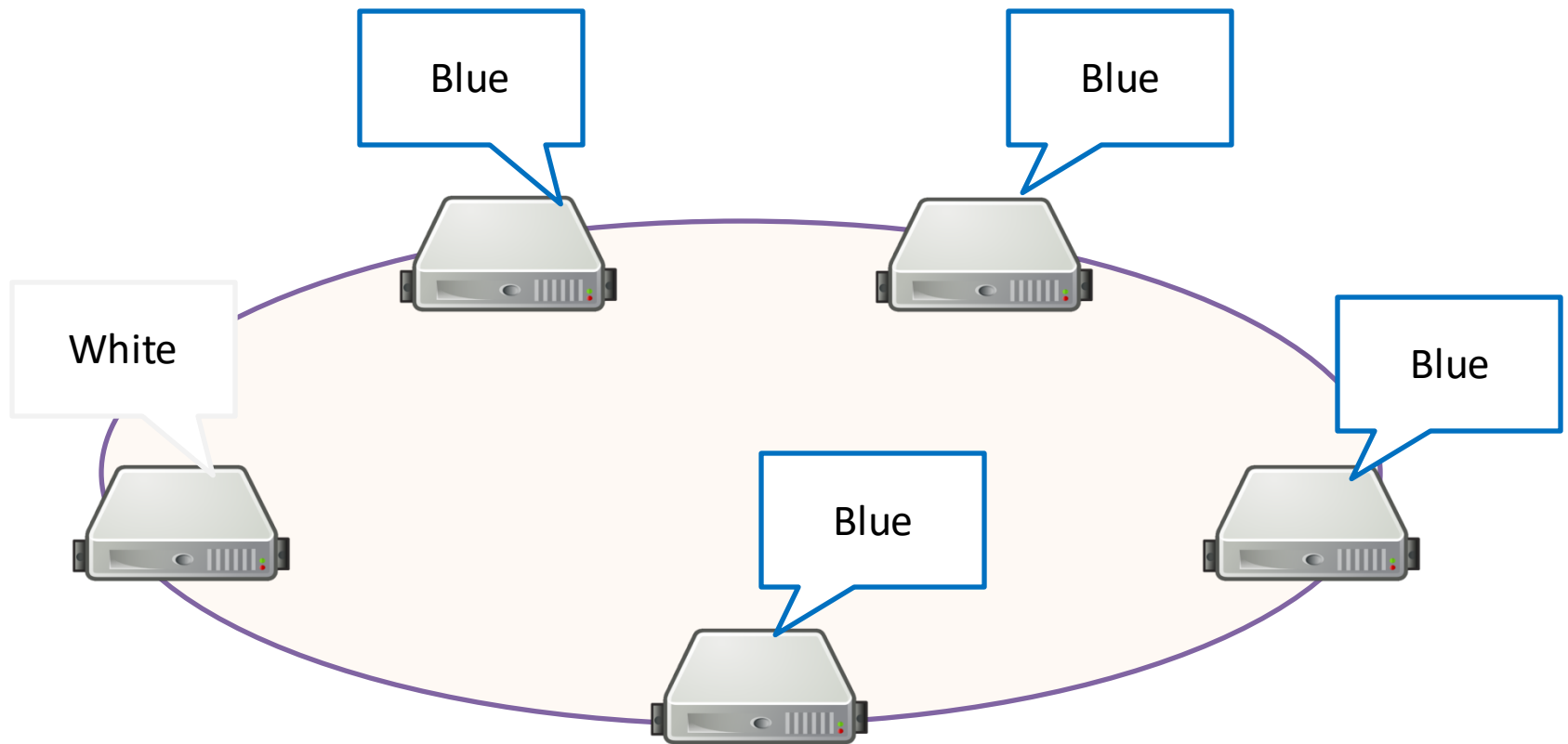
Consenso distribuido

Consenso fuerte: Todos los nodos deben concordar



Consenso distribuido

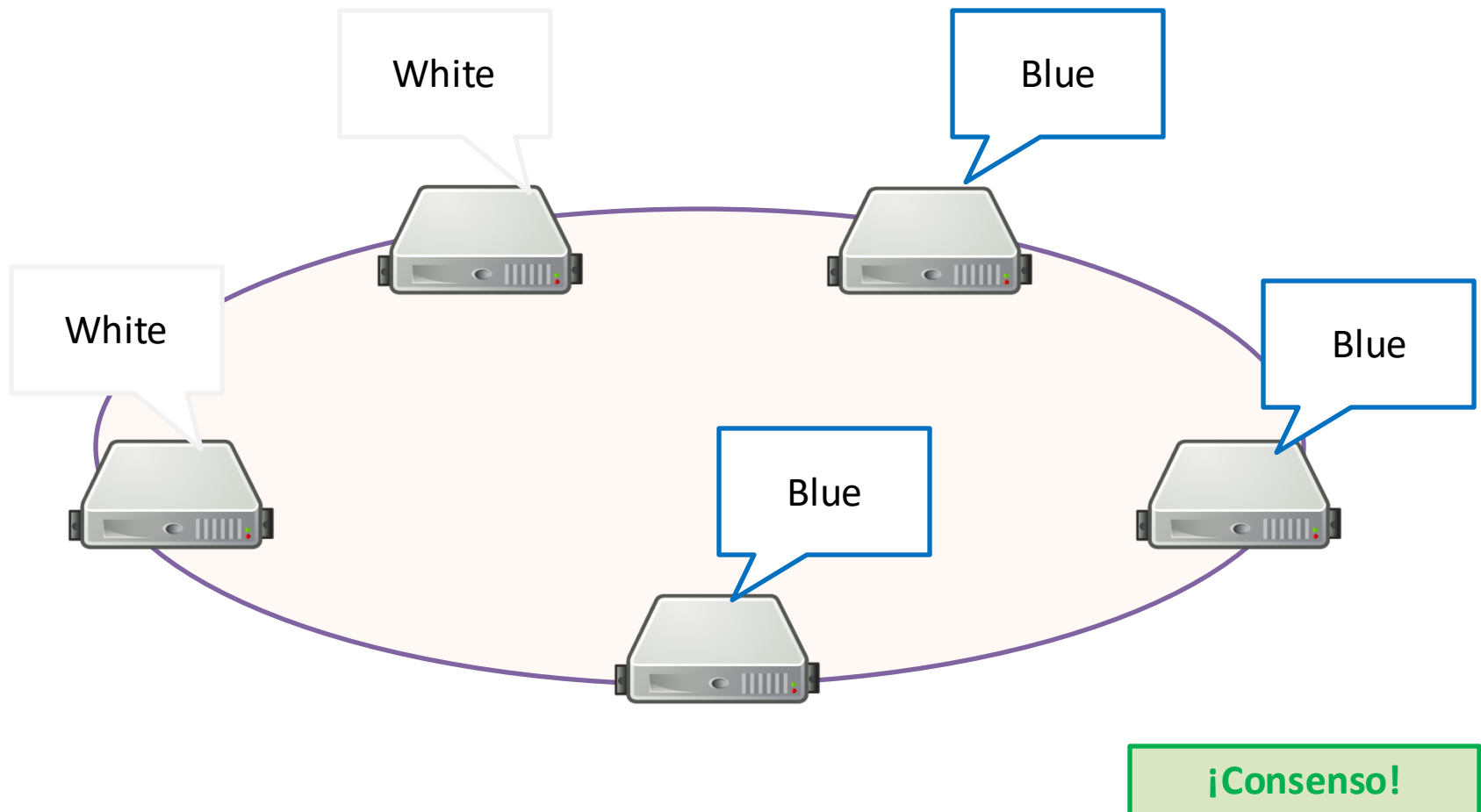
Consenso fuerte: Todos los nodos deben concordar



¡Sin consenso!

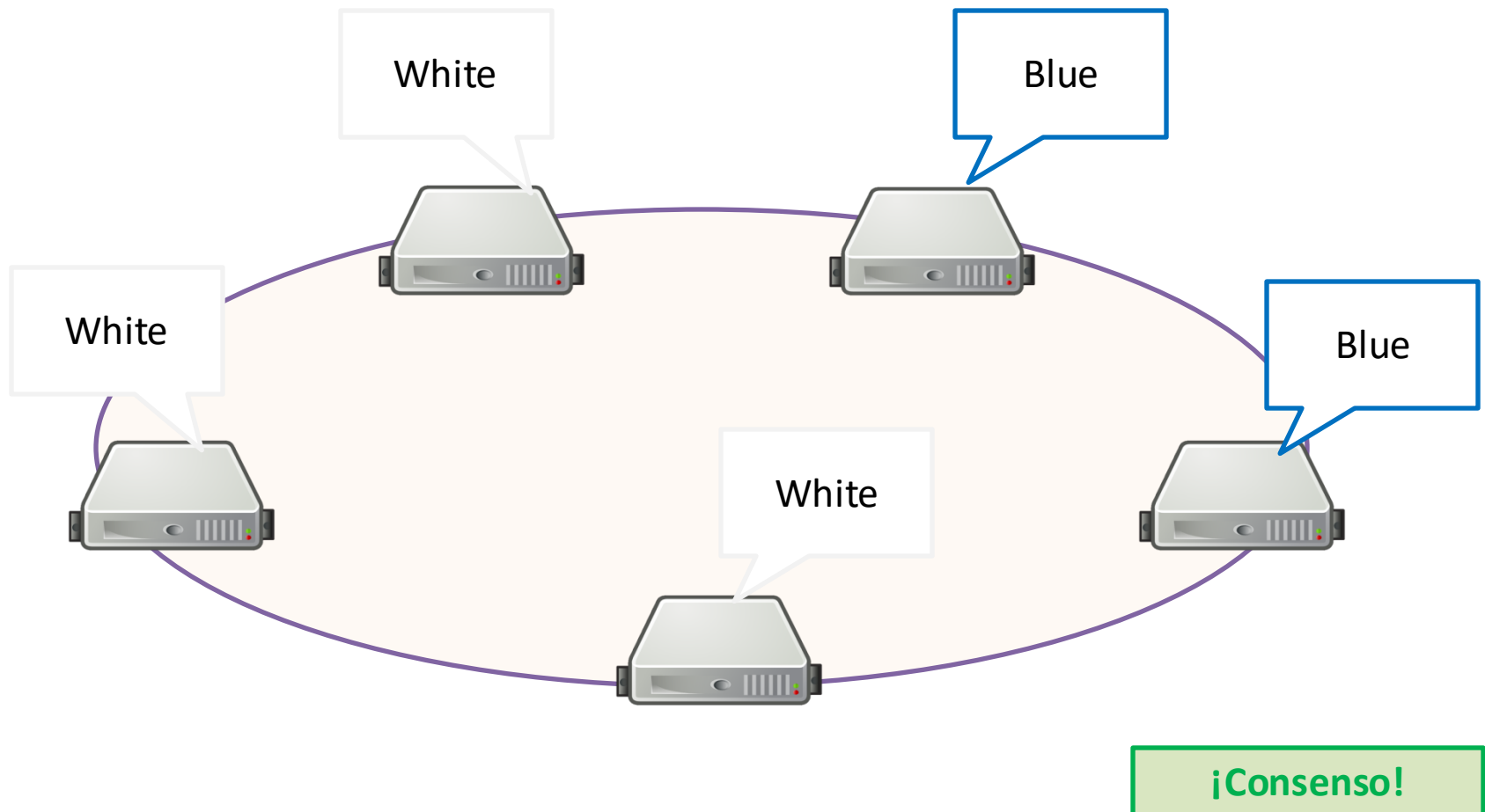
Consenso distribuido

Consenso mayoritario: Debe concordar una mayoría



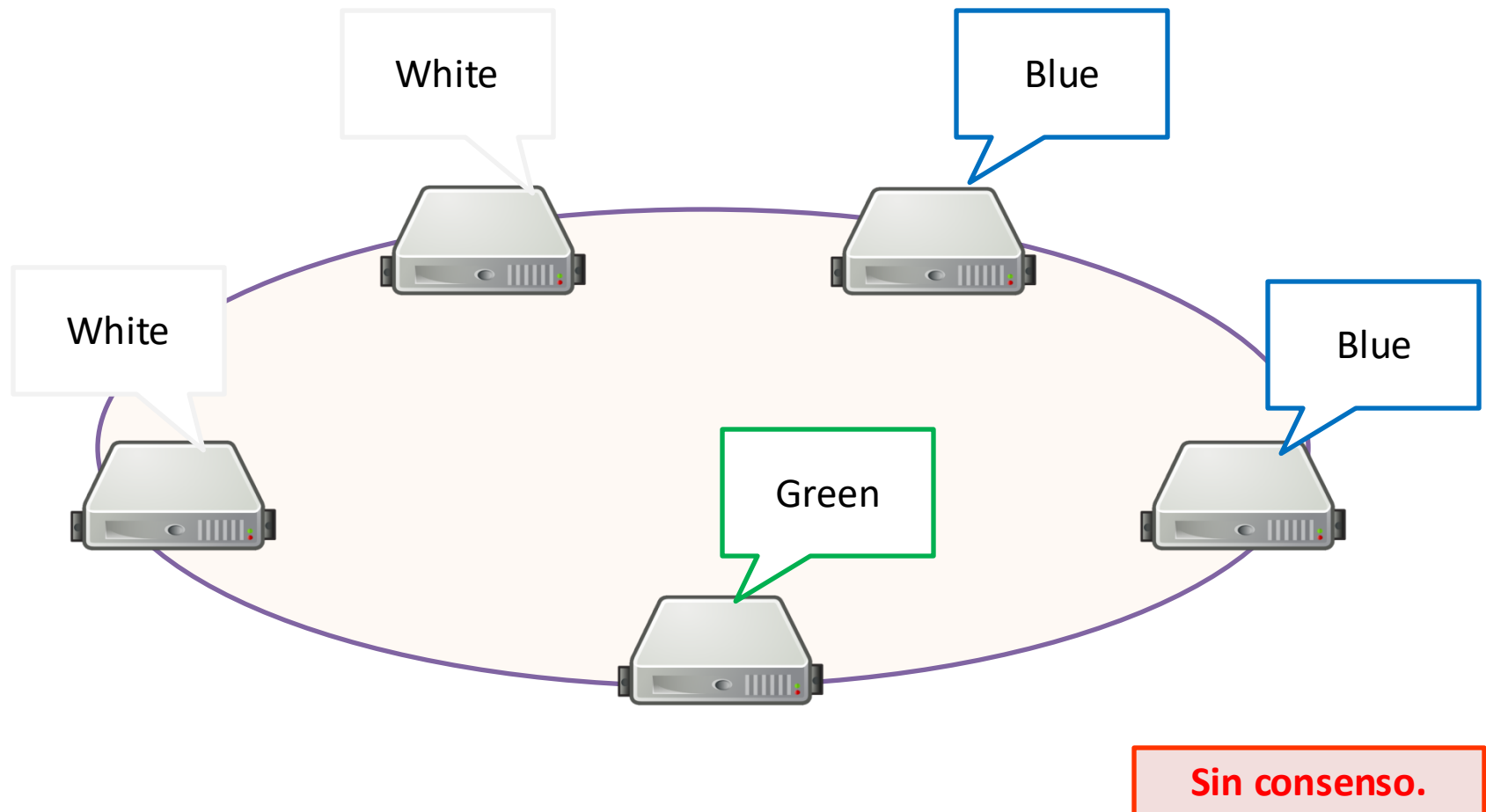
Consenso distribuido

Consenso mayoritario: Debe concordar una mayoría



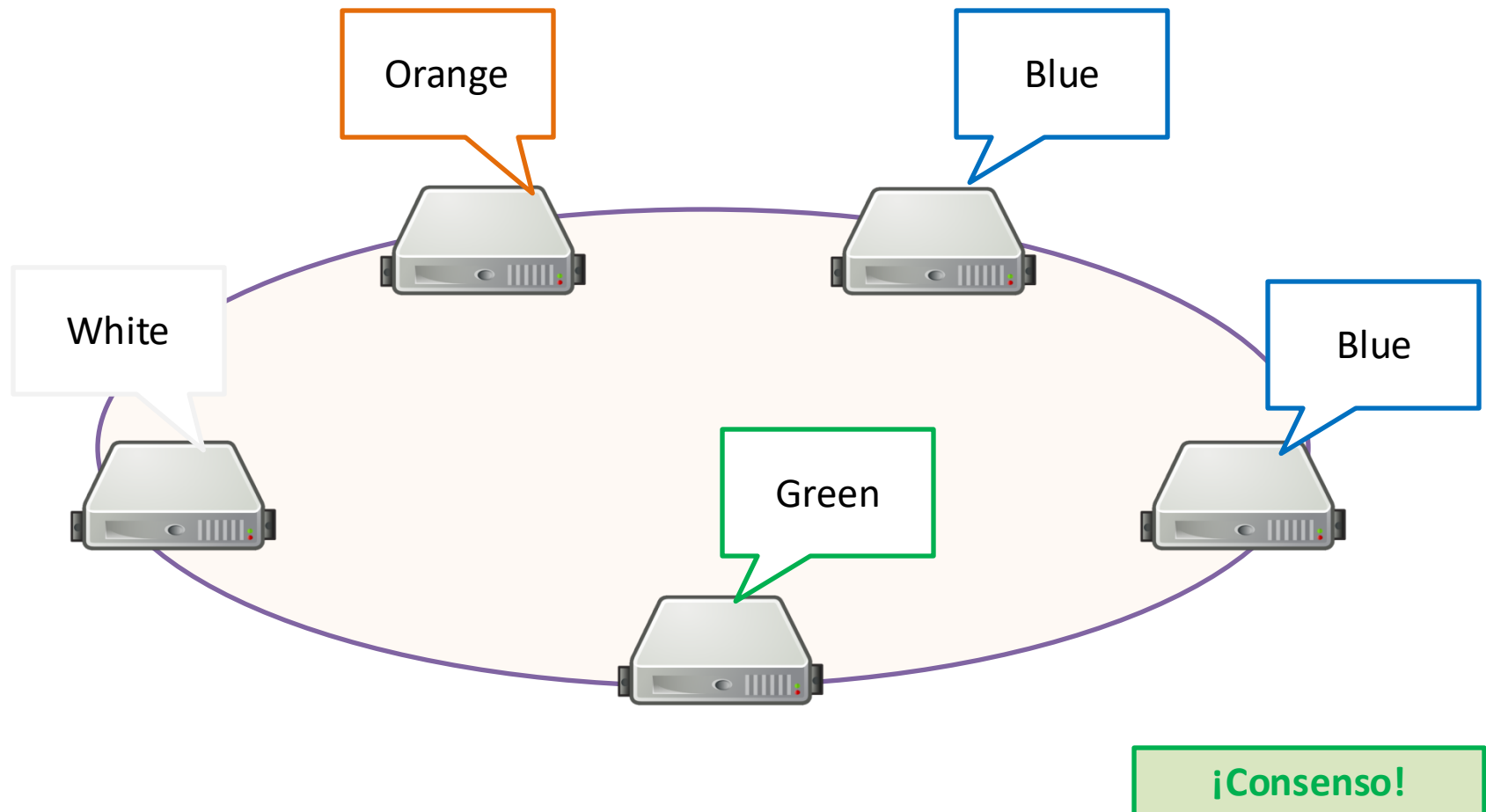
Consenso distribuido

Consenso mayoritario: Debe concordar una mayoría



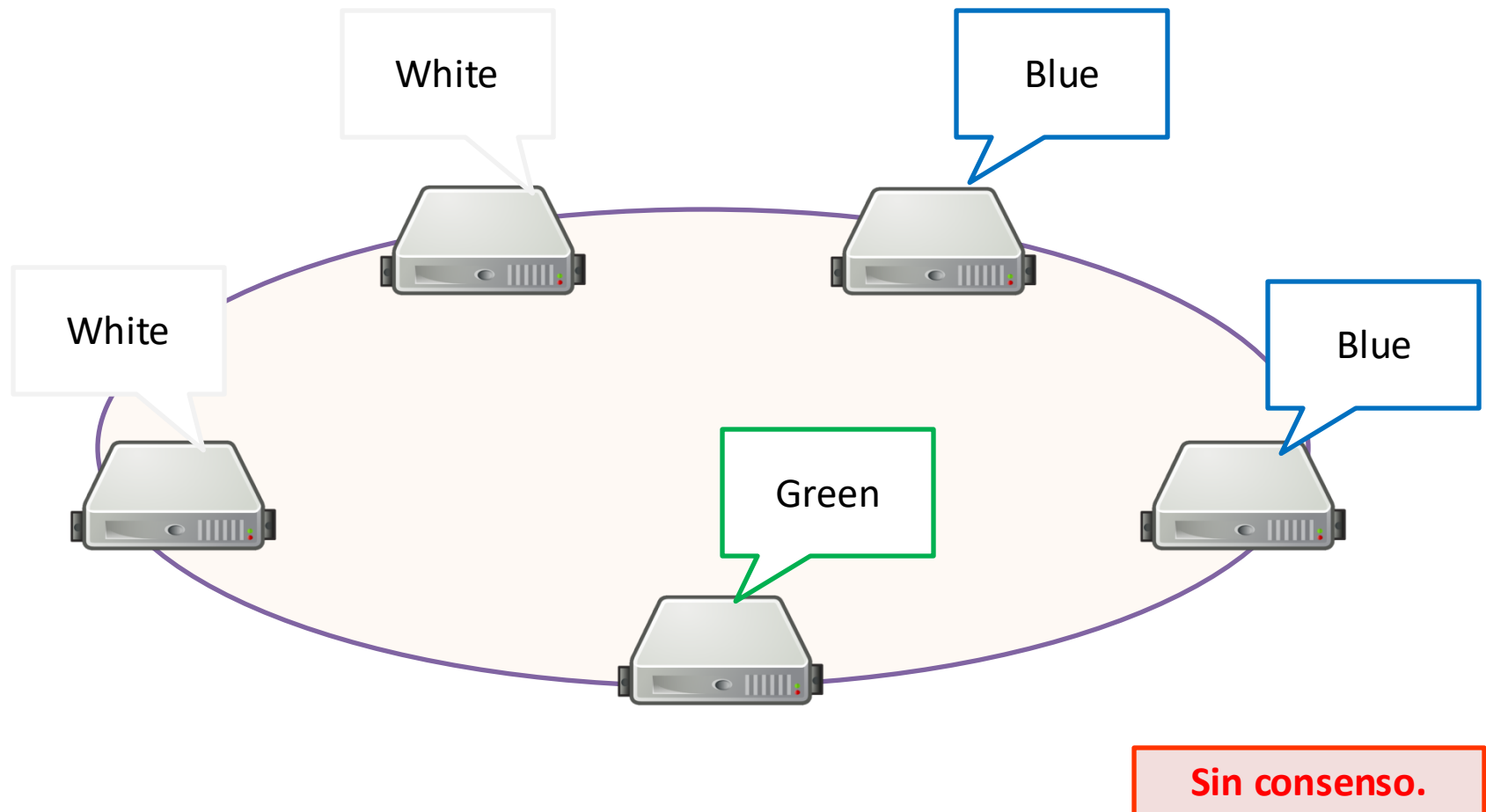
Consenso distribuido

Consenso plural: Debe concordar una pluralidad



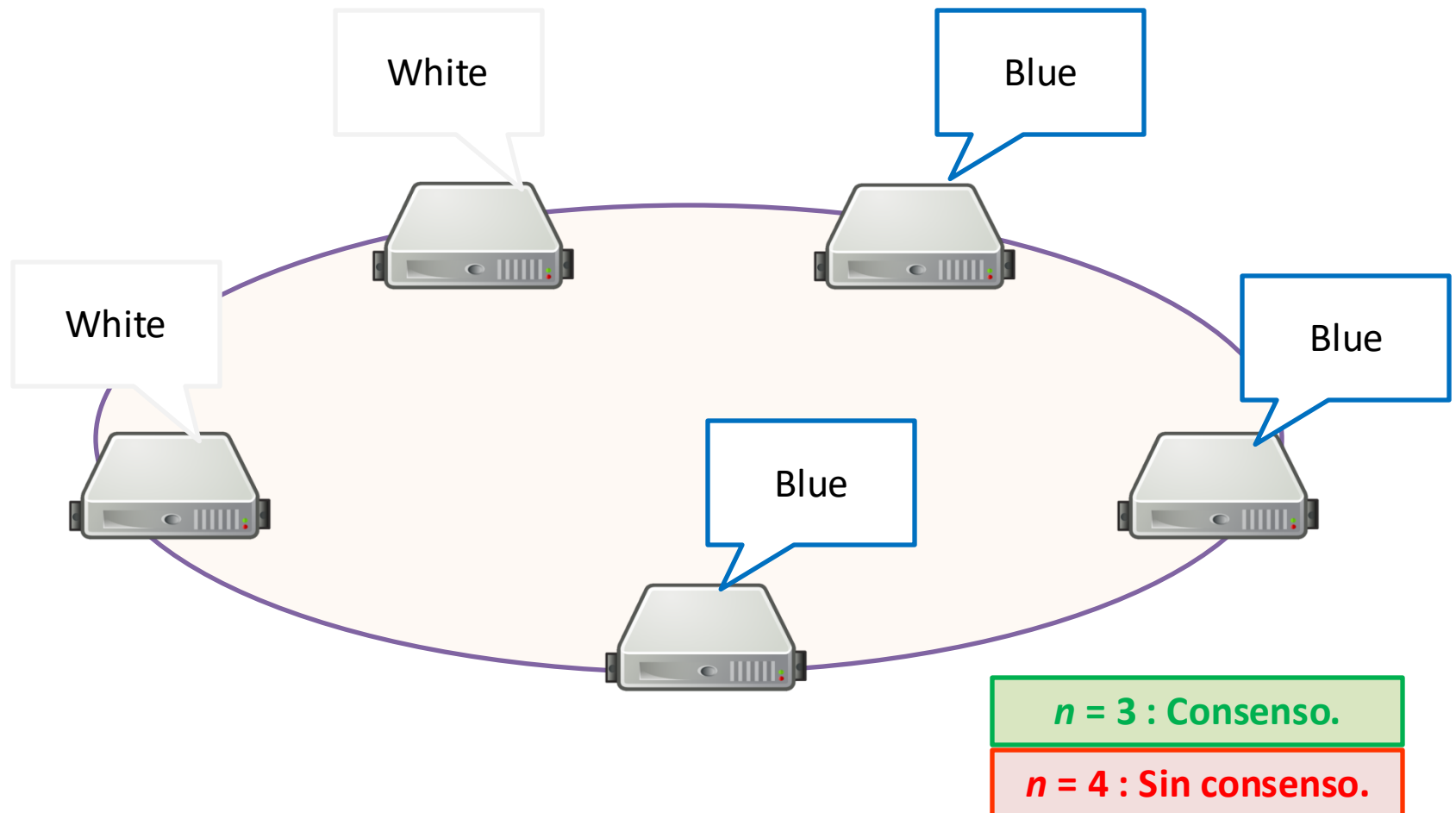
Consenso distribuido

Consenso plural: Debe concordar una pluralidad



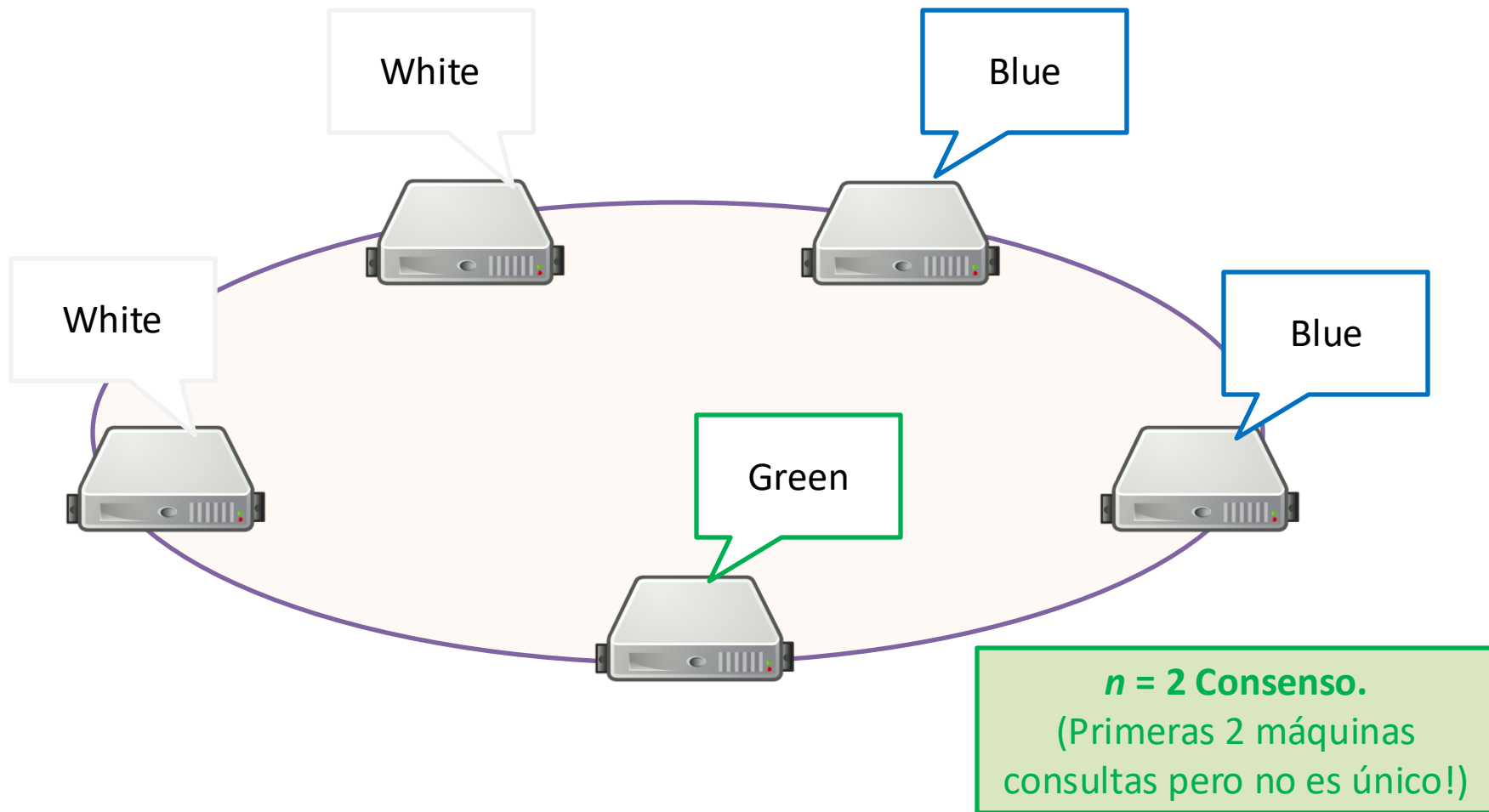
Consenso distribuido

Consenso de quorum: n nodos deben concordar



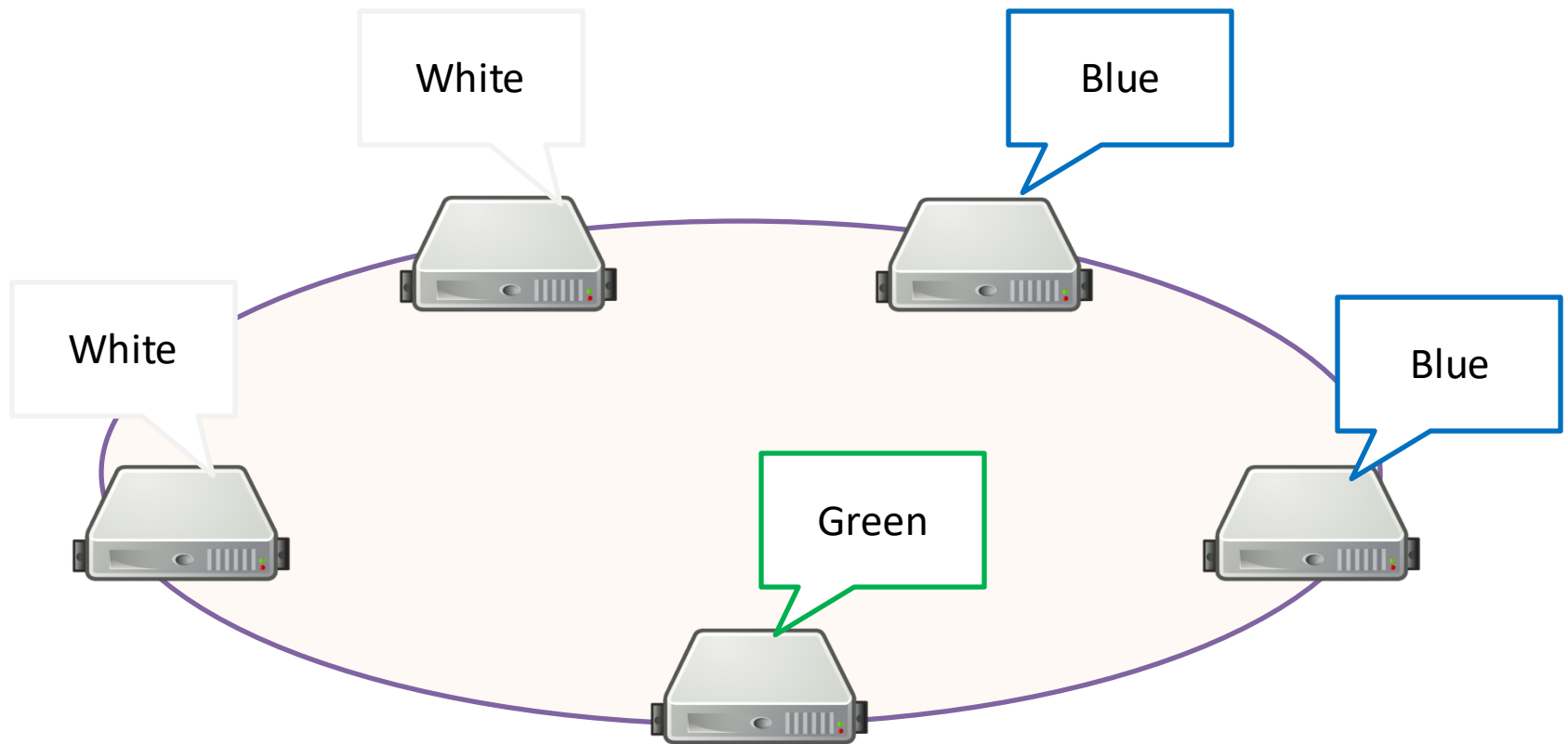
Consenso distribuido

Consenso de quorum: n nodos deben concordar



Consenso distribuido

Consenso de quorum: n nodos deben concordar

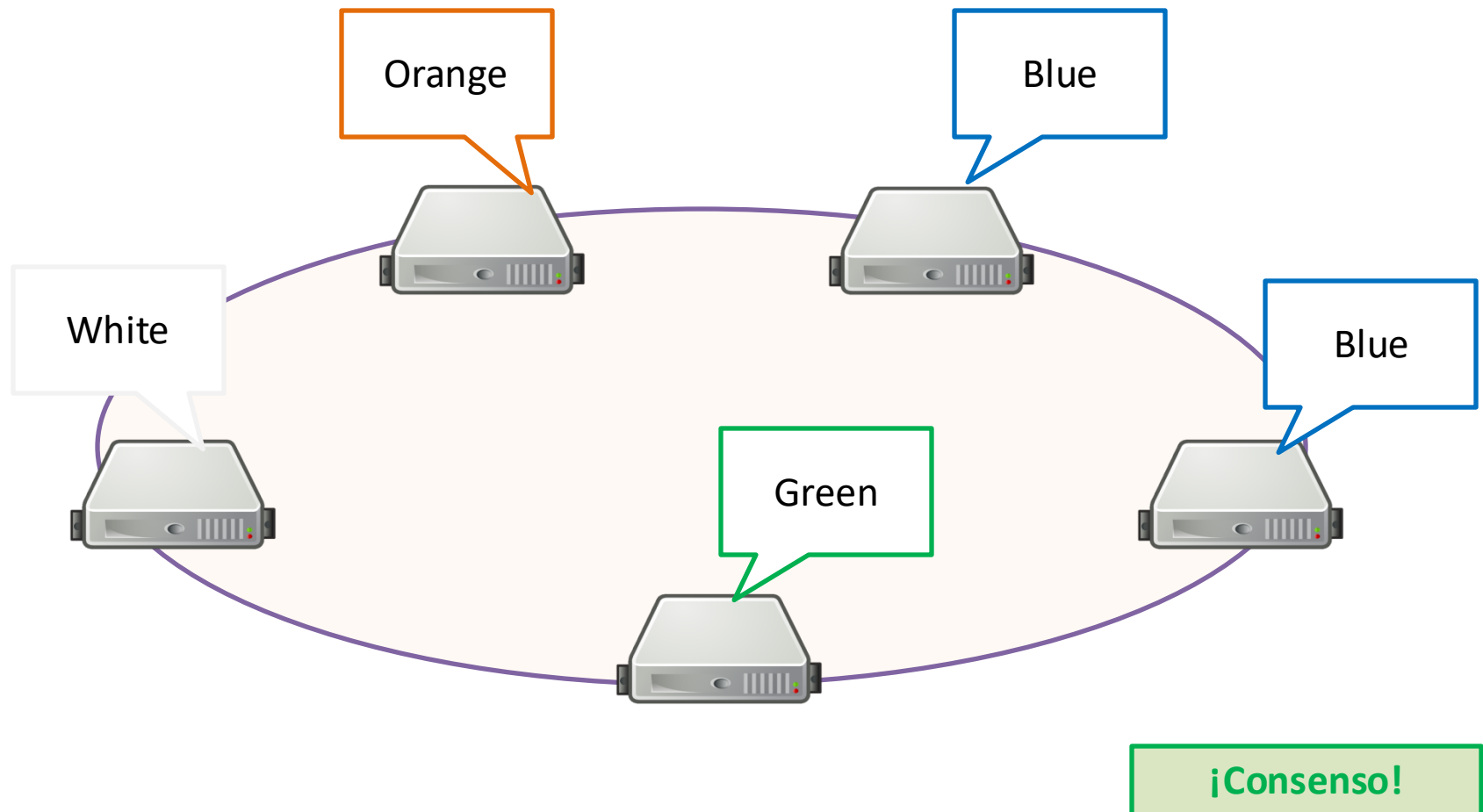


¿Valor de n necesario para un consenso único con N nodos? ?

$$n > N/2$$

Consenso distribuido

Consenso deshabilitado: Toma el primer resultado



Consenso distribuido

¿CP vs. AP?



Consenso fuerte: Todos los nodos deben concordar

CP

Consenso mayoritario: Deben concordar una mayoría

Consenso plural: Deben concordar una pluralidad

Consenso de quorum: Deben concordar n nodos

Consenso deshabilitado: Toma el primer resultado

AP

Consenso distribuido

¿La escalabilidad?



Consenso fuerte: Todos los nodos deben concordar **Más replicación**

Consenso mayoritario: Deben concordar una mayoría

Consenso plural: Deben concordar una pluralidad

Consenso de quorum: Deben concordar n nodos

Consenso deshabilitado: Toma el primer resultado

Menos replicación

Consenso distribuido

Consenso fuerte: Todos los nodos deben concordar

Consenso mayoritario: Deben concordar una mayoría

La mejor elección depende de la aplicación:

**Muchos sistemas NoSQL permiten elegir
el nivel de consenso/replicación**

Consenso de quorum: Deben concordar n nodos

Consenso deshabilitado: Toma el primer resultado